

📖 스마트 팩토리 강의 2 정리 — 제조업 머신러닝·딥러닝 기초 (의사결정나무 → CNN → RNN/LSTM → 오토인코더)

카이스트 기계공학과 유승화 교수님 강의. 제조업에서 머신러닝이 왜 필요하고 어떻게 쓰이는지를 개념부터 CNN, RNN/LSTM, 오토인코더까지 순서대로 짚어주는 강의임. 스마트팩토리 강의1이 조금 더 개론적이었다면, 이번 강의2는 "이 모델이 왜 이렇게 생겼는지"를 향아리 비유, 뉴런 비유, 주식 이동평균선 비유 등으로 계속 풀어서 설명해주는 게 특징.

✅ 5강(오토인코더) 음성 스크립트도 추가로 확보해서, 이제 0강~5강 전체가 실제 강의 음성 기반으로 정리되어 있음.

📖 다룬 내용 목록

- 0강: 머신러닝이란? — AI/ML/DL 개념, 제조업 적용목적, 예측(회귀) vs 분류, 성능평가(학습데이터 vs 검증데이터), 지도/비지도학습
- 1강: 의사결정나무(Decision Tree) & 랜덤포레스트(Random Forest) — 정보불순도, 엔트로피, 지니지수
- 2강: 인공신경망 & 딥러닝(Deep Learning) 개요 — 퍼셉트론, 활성화함수, 순전파/역전파, 과소/과대적합
- 3강: CNN(합성곱 신경망) — 합성곱, 패딩, 풀링, 플래튼
- 4강: RNN(순환신경망) & LSTM — 시계열 데이터, 그래디언트 소실, 셀 스테이트
- 5강: 오토인코더(Autoencoder) — 지도학습 vs 비지도학습, bottleneck 구조, MNIST 차원축소, 잠재공간 보간, 노이즈 제거, KAMP 사출성형기 이상탐지 사례

[0강] 머신러닝이란?

강사 소개

카이스트 기계공학과 유승화 교수. 스탠포드대 물리학 박사(2011), KAIST 물리학 학사(2004) 출신이고 본인이 직접 밝히길 "AI 자체를 전공한 사람은 아니다"라고 함. 학부/대학원 때 물리학 전공했고, 지금은 기계공학과에서 소재·구조를 멀티스케일 모델링과 컴퓨터 시뮬레이션으로 최적 설계하는 연구를 함. 최근 머신러닝이 화두가 되면서 이 알고리즘들을 기계공학·생산설계 분야에 적용해서 연구성과를 내고, 기업 자문도 하고 있다고. 그래서 "AI 전공자가 아니라 도메인 지식을 갖고 이미 개발된 AI/ML 알고리즘을 활용하는 사람"이라서 오히려 현장에서 실제로 활용하려는 사람들에게 더 도움이 될 거라고 자기소개함. 본 강의는 총 6개 (0강~5강)로 구성.

 [0강 PDF 1페이지 — 강연자 소개 슬라이드, 프로필 사진 포함]

강연자 소개



KAIST 기계공학과 유승화

스탠포드대 물리학 박사 (2011)

KAIST 물리학 학사 (2004)

소재, 구조, 공정 설계에

머신러닝 알고리즘 활용 연구/자문

M⁴ | Multiscale Mechanics &
Materials Modeling Lab

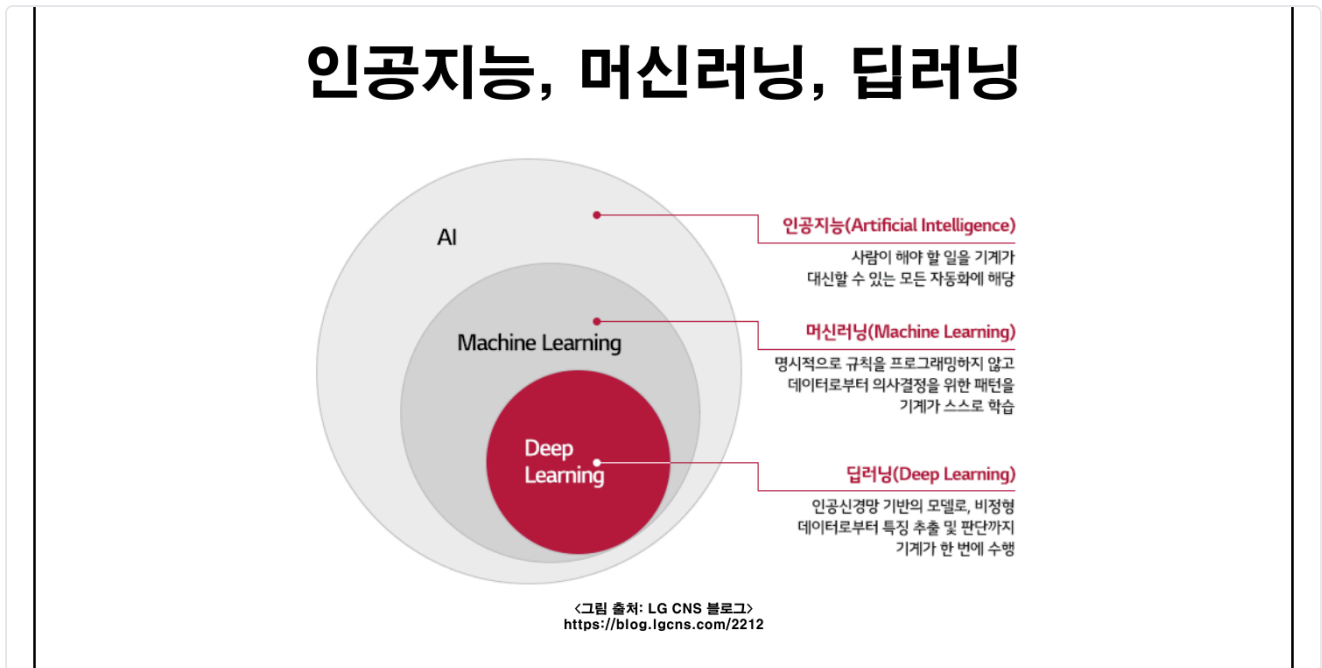
멀티스케일 역학 및 재료 모델링 연구실: <https://sites.google.com/site/seunghwalab>

인공지능 / 머신러닝 / 딥러닝 개념 구분

- **인공지능(AI)**: 사람이 해야 할 일을 기계가 대신할 수 있는 모든 자동화. 예를 들어 사진 보고 고양이/개 구분하는 것부터, 철학적 사유로 삶의 가치를 생각해보는 것까지 다 인공지능의 범주에 들어갈 수 있음. 다만 사람을 완전히 대체할 만한 "완벽한 인공지능"까지는 아직 갈 길이 멀다고 함.
- **머신러닝(ML)**: 명시적으로 규칙을 프로그래밍하지 않고, 데이터로부터 의사결정·예측·분류를 하는 학습법. 기존 온도 제어에 예로 들면, "목표온도-현재온도 차이"와 "현재온도 변화속도"를 조합해서 완벽한 규칙을 만드는 PID 제어기 같은 방식이 아니라, 물리적 원리나 제어이론에 근거하지 않고 순전히 데이터만 갖고 예측·분류·명령을 학습시키는 방법론.
- **딥러닝(DL)**: 머신러닝의 일종으로, 사람의 신경망 모델을 모사한 인공신경망 모델. 최근 발전한 GPU 등 컴퓨터 성능 덕분에 예전엔 불가능했던 아주 깊고 자유도 높은 인공신경망을 만들 수 있게 되었고, 그 덕에 비정형 데이터까지 기계가 쉽게 배우

고 예측·추론할 수 있게 됨. (이세돌·알파고 바둑 대국이 딥러닝을 유명하게 만든 계기로 언급됨)

📷 [0장 PDF 2페이지 — AI ⊃ ML ⊃ DL 벤 다이어그램, 출처: LG CNS 블로그]



본 강의는 완전한 인공지능은 다루지 않고, 머신러닝 개념 + 그중 의사결정트리/딥러닝/오토인코더 일부를 다루면서 제조업 활용과 연결하는 방식으로 구성됨.

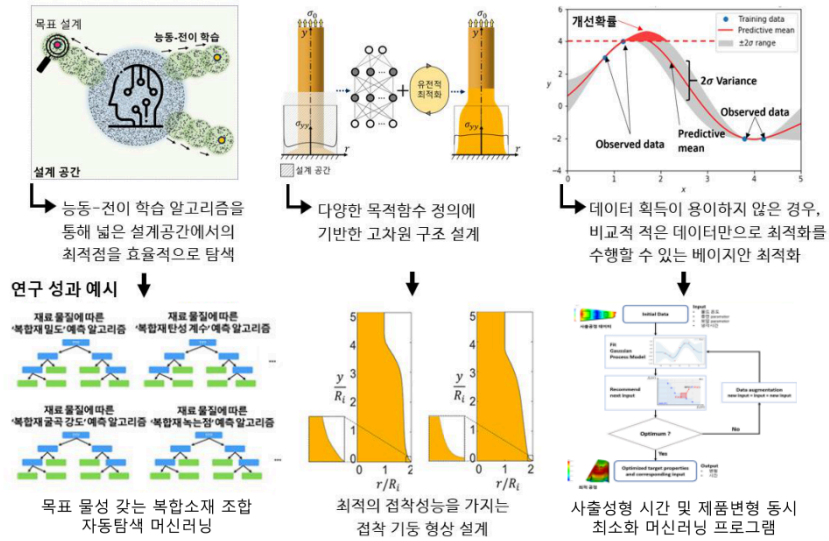
제조업의 머신러닝 적용 목적 (2가지)

목적 I — 소재 개발 / 구조(형상) 설계 / 공정 최적화

- 원하는 물리적 성질을 가진 복합재를 만들 때, 어떤 모재·강화제·화학적 첨가제·부피비율을 써야 그 물성이 나오는지, 기존엔 회사 전문가의 직관과 경험으로 디자인했지만, 이제는 데이터를 취합해서 컴퓨터가 탐색해서 찾아낼 수 있음.
- 원하는 기계적/물리적/전기적/자기적 성능을 갖는 형상이나 소자 구조를 설계할 때도, 기존엔 시뮬레이션·실험을 여러 번 해서 찾았다면, 데이터를 많이 만들고 패턴을 학습해서 최적 성능을 갖는 형상/소자를 설계 가능.
- 공정 최적화: 소재나 형상을 찾아도 실제로 만들 수 없으면 소용없음. 예를 들어 사출 성형이라면 사출 시간·압력·냉각시간 등을 합해서 가장 짧은 시간에 사출하면서도 사후 변형·균열이 없는 조건을 머신러닝 기반으로 찾을 수 있음.
- **NEW** (PDF 전용) 연구 성과 예시로 "복합재 조합 자동탐색 머신러닝"(목표 물성 갖는 복합소재 조합 탐색), "최적의 접착성능 접착 기동 설계"(다양한 목적함수 정의에 기반한 고자유도 구조설계), "베이지안 최적화"(데이터 획득이 용이하지 않을 때 비교적 적은 데이터만으로 최적화, 사출성형 시간/제품변형 동시최소화) 3가지가 그림과 함께 제시됨.

📷 [0장 PDF 2페이지 — 제조업의 머신러닝 적용목적 I: 복합재 조합 탐색, 접착기동 설계, 베이지안 최적화 그림 3종]

제조업의 머신러닝 적용목적 I

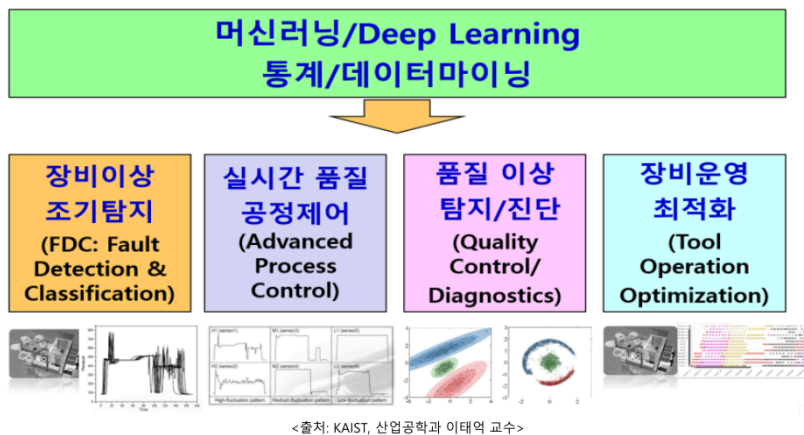


목적 II — 실시간 운영·품질·이상탐지

- 공정 중 활용되는 장비의 이상탐지, 실시간 품질·공정 제어, 품질 이상 탐지/진단, 장비운영 최적화.
- **NEW** (PDF 전용) 이 4가지가 각각 "장비이상 조기탐지(FDC: Fault Detection & Classification)", "실시간 품질 공정제어 (Advanced Process Control)", "품질 이상 탐지/진단(Quality Control/Diagnostics)", "장비운영 최적화(Tool Operation Optimization)"라는 정식 명칭으로 표기되어 있고, 각각에 대한 예시 그래프/그림이 붙어있음 (출처: KAIST 산업공학과 이태억 교수).

[0강 PDF 3페이지 — 제조업의 머신러닝 적용목적 II: FDC/APC/QC-진단/장비운영최적화 4분류 다이어그램]

제조업의 머신러닝 적용목적 II



강의 개요(총 6강): 0강 머신러닝이란? / 1강 의사결정나무(Decision Tree) / 2강 딥러닝(Deep Learning) / 3강 합성곱신경망 (CNN) / 4강 순환신경망(RNN) / 5강 오토인코더(Autoencoder)

예제 1 — SCT사 활성탄 재생 공정 (예측/회귀 예제)

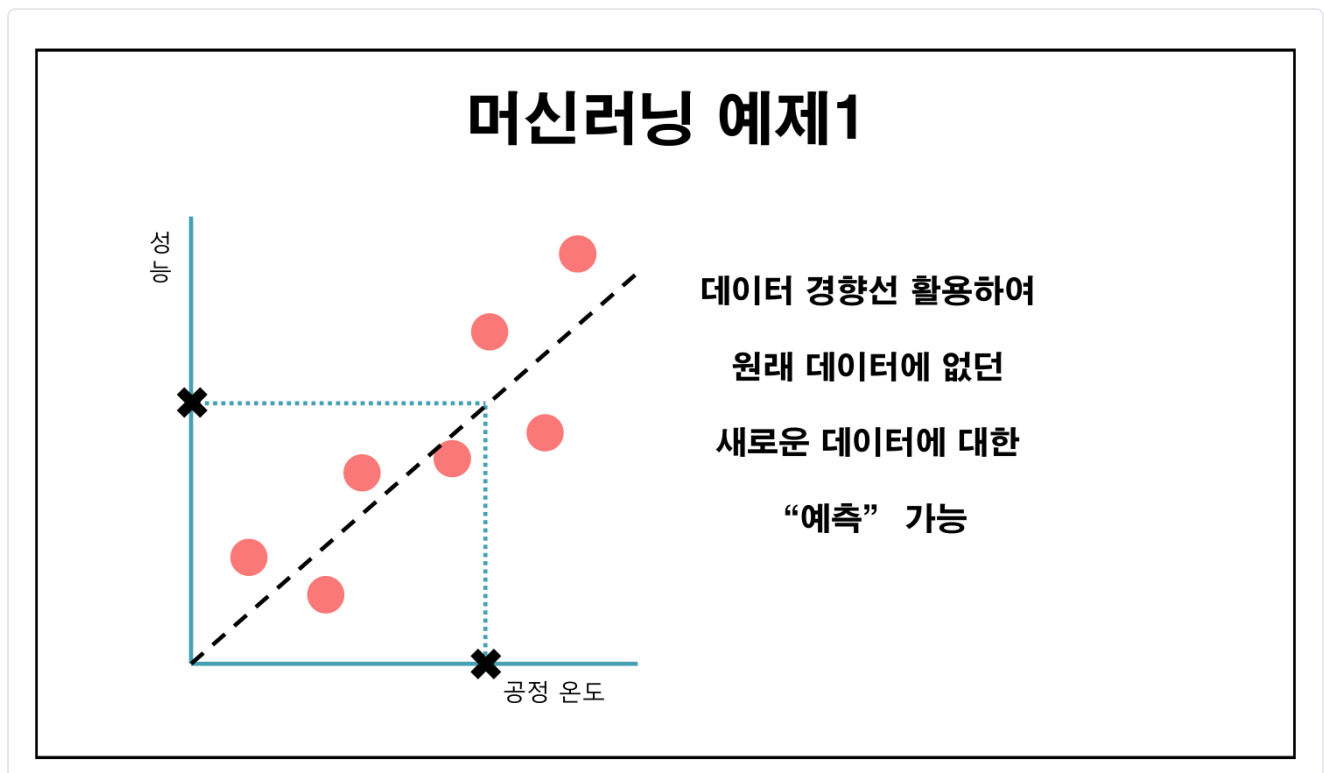
저자가 자문 갔던 회사 중 SCT(⚠ 회사명, 음성인식 오류 가능성 있음)라는 곳은 이미 흡착 성능이 떨어진 폐활성탄을 고온의 킬른(가열로)에 넣고 오래 돌려서 흡착 성능을 다시 회복시키는(재생 활성탄 제조) 공정을 하는 회사.

문제를 단순화하기 위해 "공정 온도 하나만 조절 가능하다"고 가정하고 성능 데이터를 모았다고 가정:

- 상대적으로 낮은 온도에서는 성능이 나빠서 "부적합 공정조건"으로 판단
- 데이터를 더 모아보니 온도가 좀 더 높아지면 성능이 올라가서 "사용 가능 공정" 영역이 생김
- 온도를 더 바꿔보니 성능이 더 좋아지는 지점을 발견 → 경험적으로 찾은 "최적의 공정 조건"

이 데이터에 검은 점선으로 전반적인 경향선을 그릴 수 있는데, 이것 **회귀분석**이라고 함(기존 확률·통계에서도 다루는 분석). 이 경향선이 있으면, 실제로 실험 안 해본 새로운 공정온도에 대해서도 "이 온도면 이 정도 성능이 나올 것이다"라는 **예측**이 가능해짐. 데이터가 쌓일수록 경향선도 조금씩 바뀌는데, 이런 걸 쉽게 해주는 게 머신러닝 알고리즘.

📷 [0강 PDF 4~6페이지 — 머신러닝 예제1: 공정온도-성능 산점도가 "부적합→사용가능→최적조건" 순으로 발전하는 5장의 슬라이드, 마지막엔 검은 점선 경향선과 예측 화살표(X자 표시)]



예제 2 — 아주스틸 컬러강판 공정 (분류 예제)

또 다른 자문 사례는 아주스틸(⚠ 회사명, 음성인식 오류 가능성 있음)에서 컬러 강판을 만드는 공정. 공정 순서: 세정 작업 → 적정 점도의 도료 분사 → 건조 → 품질검사(합격/불합격 판정).

가상으로 283개의 제품 데이터를 모았다고 가정(실제 PDF에도 283개로 나눔). 기준 3가지:

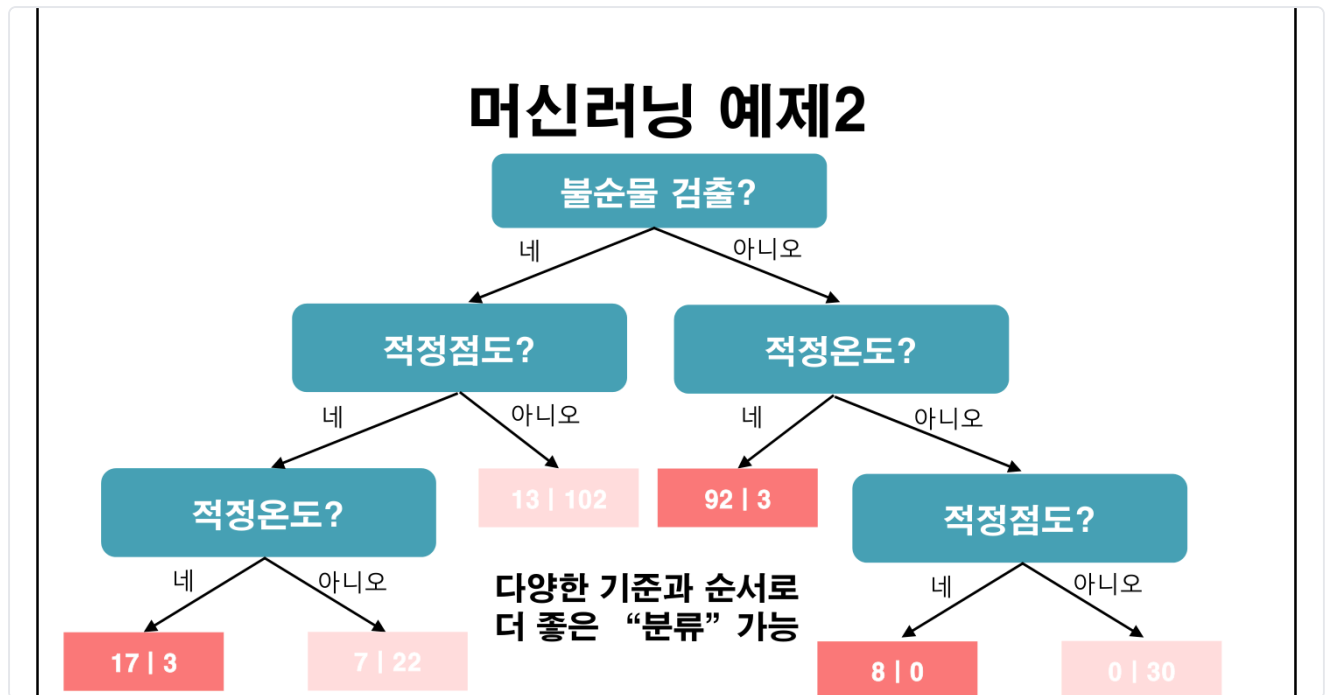
1. 적정온도(세정 시 온도가 적절했는지)
2. 불순물 검출(세정액에서 불순물이 검출됐는지)
3. 적정점도(도료 점도가 적정했는지)

먼저 **적정온도 하나만** 갖고 분류해보면: 적정온도였을 때 합격 105개/불합격 39개, 적정온도가 아니었을 때 합격 34개/불합격 125개. 즉 온도가 적정했음에도 불합격이 나오고, 적정하지 않았어도 합격이 나오는 경우가 섞여 있어서 **한 가지 기준만으로는**

완벽한 분류가 어려움. → 이래서 3가지 조건을 다 써야 하고, 그중 "어떤 조건을 먼저, 어떤 순서로 쓸지"를 제안해주는 게 **의사 결정나무(디시전트리)**.

예시로 든 트리 구조: 불순물 검출을 먼저 물어봄(가장 잘 나뉘는 기준) → 불순물 검출됐으면 적정점도를 물어봄 → 점도도 안 좋으면 온도는 안 물어보고 그냥 "불합격"으로 판단(온도를 추가로 물어봐도 합격/불합격 비율이 안 바뀌기 때문에 여기서 멈춤) → 점도가 괜찮았으면 온도까지 물어봐서 최종 분류(온도 좋음: 17개 중 합격17/불합격3, 온도 안 좋음: 7개 합격/22개 불합격) 이런 식으로 283개 데이터를 나눔.

📄 [0강 PDF 7~8페이지 — 283개 데이터 표, "적정온도?" 하나로만 나눈 트리(합105/불39, 합34/불125), 그리고 불순물검출 → 적정점도 → 적정온도 순으로 내려가는 최종 트리(각 leaf의 숫자: 17|3, 7|22, 92|3, 8|0, 0|30 등)]



완벽하게 100% 나누지 못한 이유는 ① 센서 자체 오류 가능성, ② 강판 품질에 영향 주는 요소가 이 3가지 말고도 더 많은데 일 부만 썼기 때문. 하지만 의사결정나무는 "주어진 데이터 내에서 가장 효율적으로 분류하는 기준과 순서"를 알려주는 방법론이라 는 걸 강조.

머신러닝의 핵심 역할: 예측과 분류

지금까지 든 두 예제를 요약하면:

1. 데이터가 없는 부분에 대해 기존 데이터 경향성을 파악해서 성능을 예측하는 **회귀** 기능
2. 주어진 데이터셋으로 잘 분류하고, 새 데이터가 왔을 때 어떻게 될지 예측하는 **분류** 기능

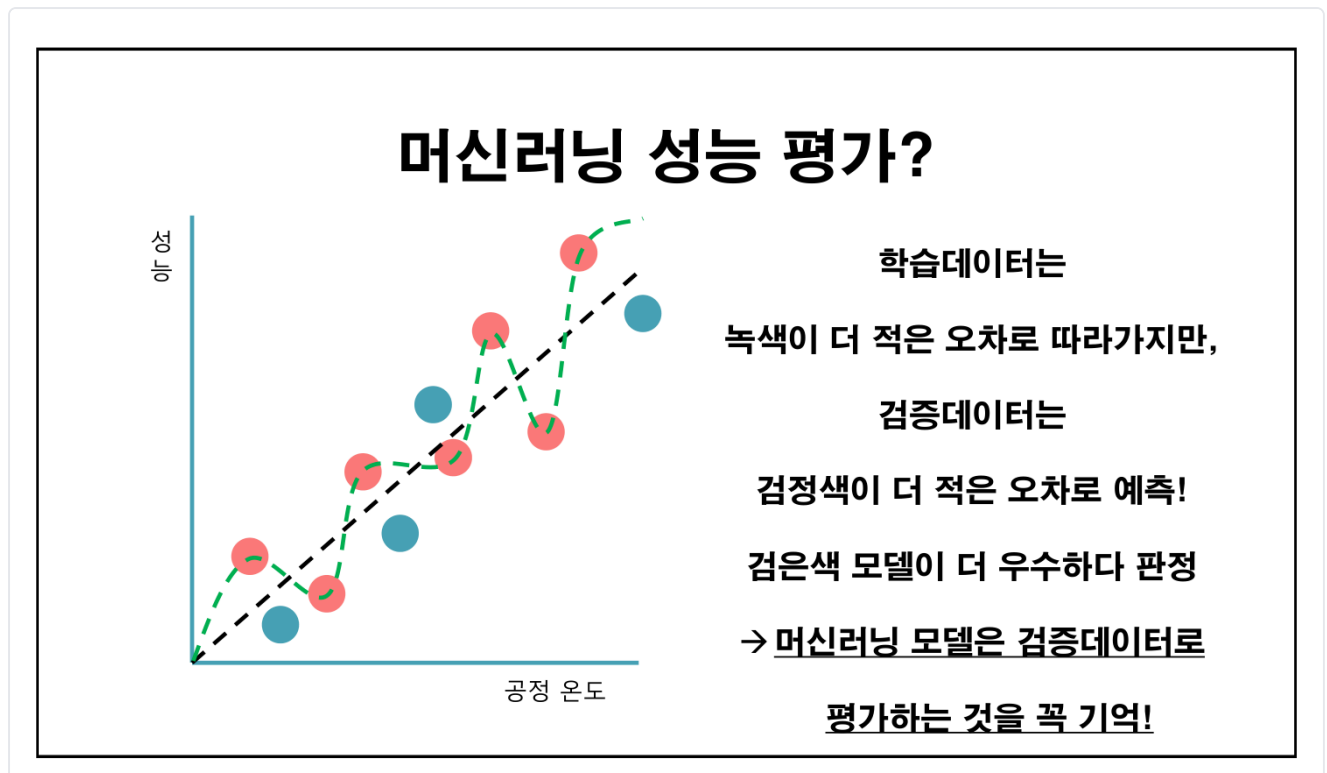
성능 평가 — 학습데이터 vs 검증(테스트)데이터

경향성을 파악하는 방법(모델)은 여러 개일 수 있음. 예를 들어 같은 데이터에 대해 오차는 좀 있지만 완전한 직선 모델 vs 오차 없 이 거의 완벽하게 따라가는 꼬불꼬불한 곡선 모델, 둘 중 뭐가 더 좋은지는 사람마다 "취향"으로 판단할 게 아니라 **확실한 성능평 가 기준**이 있음.

- 경향성 파악에 쓰인 원래 데이터 = **학습 데이터(training data)**
- 경향성 파악에 전혀 안 쓰인 새로운 데이터 = **검증(test) 데이터**

검증 데이터로 두 모델(검은 직선 vs 녹색 꼬불꼬불한 곡선)을 비교해보면: 녹색 모델은 학습데이터는 거의 완벽하게 따라갔지만, 검증데이터(새 파란 점들)에 대해서는 오차가 컸음. 검은 직선 모델은 학습데이터에서는 오차가 좀 있었지만 검증데이터에 대해서는 오히려 오차가 더 적었음. → **머신러닝의 성능평가는 학습데이터를 얼마나 잘 따라가느냐가 아니라, 새로운(검증) 데이터를 얼마나 잘 예측하느냐로 해야 한다**는 게 핵심 메시지. (이게 사실상 과적합/오버피팅 개념을 처음 소개하는 부분이고, 2강에서 "과소적합/과대적합" 용어로 다시 정식화됨)

📷 [0강 PDF 9~12페이지 — 검은 직선 vs 녹색 곡선이 학습데이터를 따라가는 비교, 그리고 파란 점(검증데이터)과 비교해서 검은 직선이 실제로 더 우수함을 보여주는 4장의 슬라이드]



일반화하면: 실제로는 "성능 = f(공정온도)" 처럼 단순하지 않고 "성능 = f(공정온도, 공정압력, 공정시간, 냉각속도, 냉각시간, ...)" 처럼 다변수 함수임. (알파고 비유: 성능→승리확률, 입력→지금까지 놓인 바둑돌+학습한 기보, 이것도 유사한 원리라고 함). 이 함수 f()를 구하는 방법으로 선형회귀, 심층신경망, 의사결정나무 등이 있고, **사용자가 할 일은 주어진 문제에 대해 검증데이터를 가장 잘 예측하는 모델을 선택하는 것.**

요약

- 제조업의 머신러닝 = 제조데이터 + 모델 → 예측 혹은 분류
- 제조데이터: 측정값, 전문가 경험 등을 수치화한 데이터
- 모델: 선형회귀, 심층신경망, 의사결정나무, 서포트벡터머신 등
- 평가: 검증데이터로 예측/분류 성능 평가
- **지도학습**: 정답(합격/불합격 등)을 가르쳐주고 모델을 학습하는 것
- **비지도학습**: 정답 없이 시계열 데이터에서 이상 탐지를 하거나, 비슷한 것끼리 묶는(군집화) 것 — 정답 없이 분류하거나 차원 축소 하는 것

[1강] 의사결정나무(Decision Tree) & 랜덤포레스트(Random Forest)

도입 — 왜 필요한가

0강 예제2(283개 제품데이터, 적정온도/불순물검출/적정점도 3가지 기준)를 다시 가져와서, "한 가지 기준만으로는 품질기준 예측이 어려운, 대부분의 제조 공정"에 대해 어떻게 분류할지 → 의사결정나무, 랜덤포레스트. 그 작동원리는 **정보 불순도 최소화**.

정보 불순도(Impurity) — 향아리 비유

향아리 1, 2, 3에 각각 공 10개가 들어있다고 가정:

- 향아리 1: 파란공 10개 (순도 100%)
- 향아리 2: 파란공 5개 + 빨간공 5개 (불순도 높음)
- 향아리 3: 빨간공 10개 (순도 100%)

이걸 제조공정으로 치환하면, 향아리1·3은 데이터가 전부 합격(혹은 전부 불합격)인 상태, 향아리2는 합격/불합격이 반반 섞인 상태라고 생각하면 됨. 이 "불순도"를 숫자로 정확히 표현할 수 있으면, 불순도를 최소화하는 기준으로 데이터를 분류할 수 있게 됨.
→ 이걸 수치화하는 게 **정보 엔트로피**와 **지니지수**.

정보 엔트로피(무질서도)

$$H(\text{정보}) = -\sum_{i=1}^c p_i \times \log(p_i)$$

- c : 사건의 개수, p_i : 사건 x_i 가 일어날 확률, $I(x_i)$: 사건 x_i 의 정보량
- 정보량 $I(x_i) = -\log(1/p_i)$ — 확률이 낮은 사건일수록 정보량이 높고, 흔한 사건(확률이 1에 가까움)은 정보량이 거의 없음.
- 비유: 회사 10명 전원이 사람이다 → 확률 1(100%) → 정보량 0 (아무 정보 아님)
- 회사 100명 중 1명이 키 2m 넘음 → 확률 1/100로 낮음 → 정보량 높음(특별한 특성)

향아리 2 계산: 빨간공 뽑을 확률 1/2, 정보량 $\log(2)$. 파란공도 동일. 정보엔트로피 = $1/2 \times \log(2) + 1/2 \times \log(2) = \log(2)$

향아리 1 계산: 파란공 확률 1, 빨간공 확률 0. 정보엔트로피 = $0 + 1 \times \log(1) = 0$ ($\log(1)=0$)

분류 기준 좋고 나쁨 판단: 향아리2(10개 공, 정보엔트로피= $\log(2)$)를

- "붉은색이냐?"로 나누면 → 왼쪽 빨강5개(엔트로피 0), 오른쪽 파랑5개(엔트로피 0) → 합계 0 → **무질서도가 $\log(2)$ 에서 0으로**

줄어들 → 좋은 분류 기준

- "원형이냐?"로 나누면 → 왼쪽엔 모든 공 10개(파랑5+빨강5, 엔트로피 $\log(2)$), 오른쪽엔 아무것도 없음(엔트로피 0) → 합계 $\log(2)$ → **원래랑 무질서도 변화가 전혀 없음 → 의미없는 분류 기준**

지니지수(Gini index)

$$G(\text{지니지수}) = 1 - \sum_{i=1}^c p_i^2$$

로그가 없어서 엔트로피보다 계산이 간단함. 지니지수가 높을수록 무질서도가 크고, 낮을수록 순도가 높음(둘 다 같은 방향으로 움직이는 지표).

- 향아리 2: $1 - (1/2)^2 - (1/2)^2 = 1 - 0.25 - 0.25 = 0.5$
- 향아리 1: $1 - 0^2 - 1^2 = 0$

강의에서는 지니지수 기준으로 설명을 진행하지만, 정보 엔트로피를 써도 유사한 결과가 나온다고 함.

의사결정나무 구축 — 컬러강판 283개 데이터 예시

의사결정나무(decision tree): 특정 기준(질문)에 따라 데이터를 구분하는 모델. 스무고개 하듯이 어떤 질문을 먼저 물어서 분류를 시작하고, 적절한 질문 순서에 따라 데이터를 나눔.

1단계: 세 기준(적정온도, 불순물검출, 적정점도) 각각으로 나눴을 때 지니지수를 계산 → 적정온도 지니지수=0.364, 불순물검출 지니지수=0.360, 적정점도 지니지수=0.381. **불순물검출의 지니지수가 가장 낮으므로** 이걸 트리 최상단(첫 질문)으로 배치. 불순물검출 "네"→164개 샘플, "아니오"→133개 샘플.

2단계: 164개 샘플에 대해 나머지 두 기준(적정온도, 적정점도)의 지니지수를 다시 계산 → 적정온도=0.3, 적정점도=0.29. **적정점도가 더 낮으므로** 두 번째 질문으로 선택. 적정점도 "네"→49개 샘플(합격24/불합격25), "아니오"→115개 샘플(합격13/불합격102).

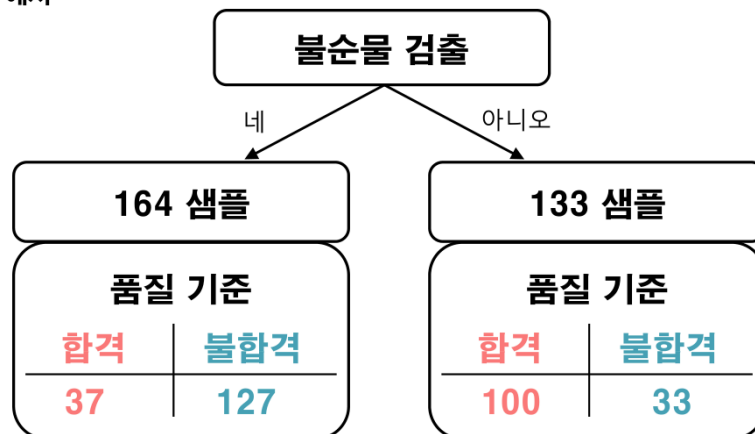
가지치기 중단 규칙: 115개 샘플의 지니지수가 0.2인데, 이걸 위 단계(적정점도 분리 전체)의 지니지수 0.29보다 이미 낮아진 상태라서, **여기서는 더 이상 가치를 추가하지 않음**. 반면 49개 샘플처럼 합격/불합격이 거의 반반으로 나뉜 경우(지니지수가 여전히 높음)는 남은 기준(적정온도)으로 한 번 더 분류를 진행.

이 과정을 반복하면서(불순물검출 → 적정점도/적정온도 → ...) 최종 의사결정나무가 완성됨. 최종 leaf 값들: 17|3(합격|불합격), 7|22, 92|3, 8|0, 0|30.

📷 [1장 PDF 6~11페이지 — 283개 데이터표, 지니지수 계산 예시(??→불순물검출 배치→164/133샘플 분기→적정온도/적정점도 지니지수 비교→49/115샘플→최종 트리)]

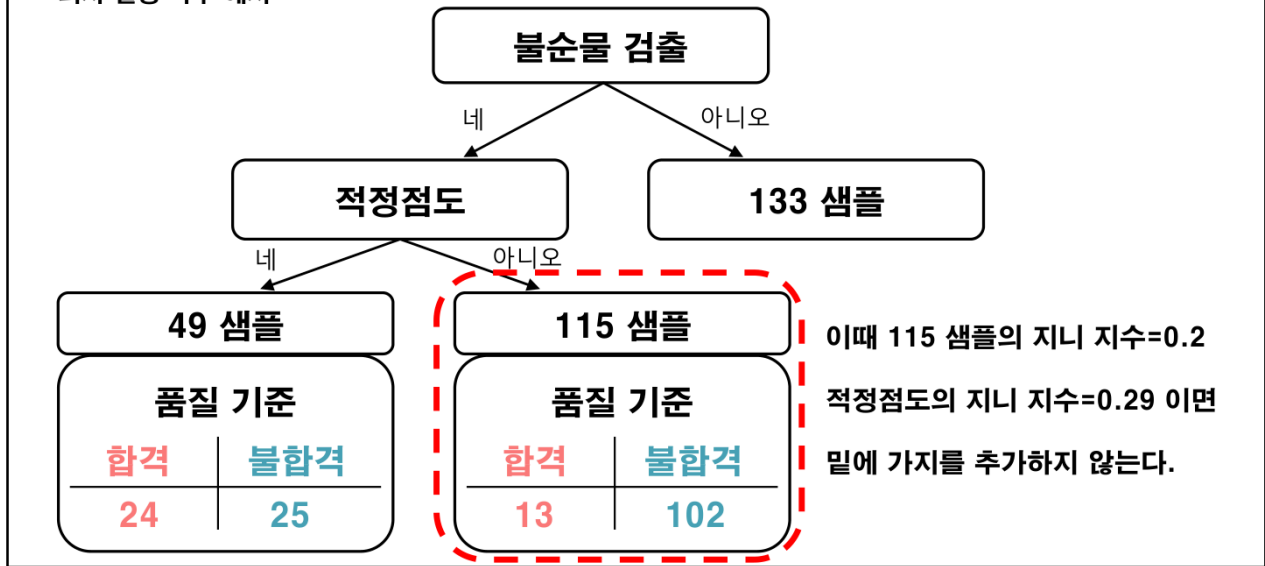
의사 결정 나무란?

의사 결정 나무 예시



의사 결정 나무란?

의사 결정 나무 예시



숫자형/등급형 데이터일 때

- **숫자 데이터**(예: 압력 155, 180, 190, 220, 225 [Mpa]에 따라 품질기준 통과여부): 인접한 두 값의 **중간값**을 계산(167.5, 185, 205, 222.5)해서, 그 중간값을 기준으로 "이상/이하"로 나눠 지니지수를 계산하는 방식으로 진행.
- **등급 데이터**(예: 재료 품질 1등급/2등급/3등급): "등급 ≤ 1", "등급 ≤ 2", "등급 ≤ 3", "등급 ≤ 4" 처럼 각 등급 기준마다 지니지수를 계산.

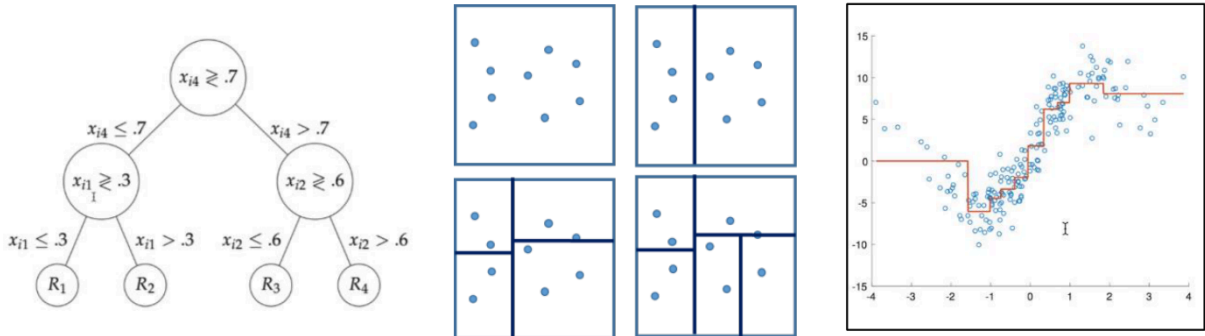
의사결정나무로 회귀도 가능하지만...

의사결정나무를 회귀에도 쓸 수는 있음(예: 압력·온도·점도가 수치로 나올 때, $X_1 < 0.3$ 이냐, $X_4 < 0.7$ 이냐 등으로 분기해서 최종 숫자를 예측). 하지만 **회귀 성능은 다른 머신러닝 모델에 비해 좋은 편이 아니어서 많이 추천되지는 않음**이라고 명확히 언급.

📷 [1강 PDF 14페이지 — 계단식 회귀(step function) 그래프와 트리 다이어그램 예시, sklearn 문서에서 흔히 보는 그 그림]

의사 결정 나무란?

의사 결정 나무를 이용하여 회귀를 진행할 수 도 있다.



의사결정나무 장단점

장점

- 논리 데이터(categorical, 예/아니오)와 숫자 데이터(numeric)를 잘 합침
- 수렴 문제 없이 속도가 빠름
- 요소의 중요도(어떤 기준이 지니지수를 얼마나 낮추는지)를 수치로 파악 가능
- 데이터 해석이 직관적임

단점

- 학습 데이터는 잘 맞추지만 새 데이터에 대한 예측이 정확하지 않음(과적합, overfitting에 취약)
- 데이터가 노이즈나 확률분포를 가질 때(예: 온도 센서가 살짝 이상해서 실제와 다른 값을 준 경우) 활용이 비효율적

랜덤포레스트

의사결정나무의 단점(과적합)을 극복하려고, 여러 개의 의사결정나무를 생성해서 각 나무의 예측을 조합해 결론을 내리는 분석.

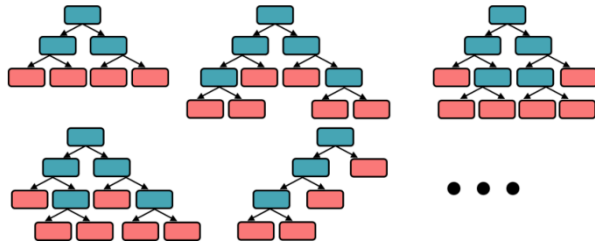
방법: 전체 데이터에서 공정 조건 중 일부(예: 4가지 중 2개)와 제품(샘플) 중 일부를 **랜덤하게 추출**해서 의사결정나무를 하나 만들고, 또 다른 조합으로 또 하나 만들고... 이걸 반복(예: 100개의 나무). **몇 개의 데이터·몇 개의 기준을 뺀을지는 하이퍼파라미터** (사용자가 정함). 예측할 때는 각 나무가 "합격"이나 "불합격"이나 한 표씩 행사하고, 예를 들어 100개 나무 중 84개가 "합격", 16개가 "불합격"이라고 예측하면 최종적으로 "합격"으로 예측.

랜덤포레스트도 의사결정나무처럼 지니지수 기반 중요도 파악이 가능해서, "어떤 공정 조건이 최종 품질에 가장 중요한지" 알 수 있는 유용한 모델.

📷 [1강 PDF 16~17페이지 — 데이터 랜덤추출→여러 의사결정나무 생성→각 나무의 투표(84표 vs 16표)로 최종 예측하는 과정 다이어그램]

랜덤 포레스트란?

각 의사 결정 나무는 1표씩 행사할 수 있음



	공정 A	공정 B	공정 C	공정 D	품질 기준 통과
test1	네	아니오	네	아니오	아니오



모든 의사 결정 나무에 대해 예측 수행

합격을 예측한 나무 수	불합격을 예측한 나무 수
84	16



예측: 합격

적용 사례 (KAMP 데이터셋)

NEW (PDF 전용, 출처 각주 포함)

- **의사결정나무 사례 — 살균기 AI 데이터셋**: 인풋(살균기 A/B의 살균상태, 살균온도), 아웃풋(양품/불량), 데이터 21만794개(학습70%/테스트30%). Confusion matrix: 237 / 196 / 1 / 2,381 (⚠️ 스크립트 음성으로는 "정상 2300개, 비정상 196개... 불합격이라 예측한 것 중 정상 1개, 비정상 237개"라고 말했는데 PDF 표의 정확한 행/열 배치와 숫자 순서가 스크립트 설명과 완전히 일치하지 않아 보임 — 실제 숫자는 PDF의 237/196/1/2381이 맞다고 보고, 어느 셀이 TP/FP/FN/TN인지는 원본 슬라이드 참고 필요). F1 score: 0.9603

- **랜덤포레스트 사례 — 사출성형 공정 정상/비정상 데이터셋**: 인풋은 사출시간, 스크류위치, 스크류 최대/평균속도, 형체력, 충전시간, 보압절환위치 등 다수 변수. 데이터 88만6,227개(학습70%/테스트30%). Confusion matrix: 56 / 2 / 0 / 357. F1 score: 0.98

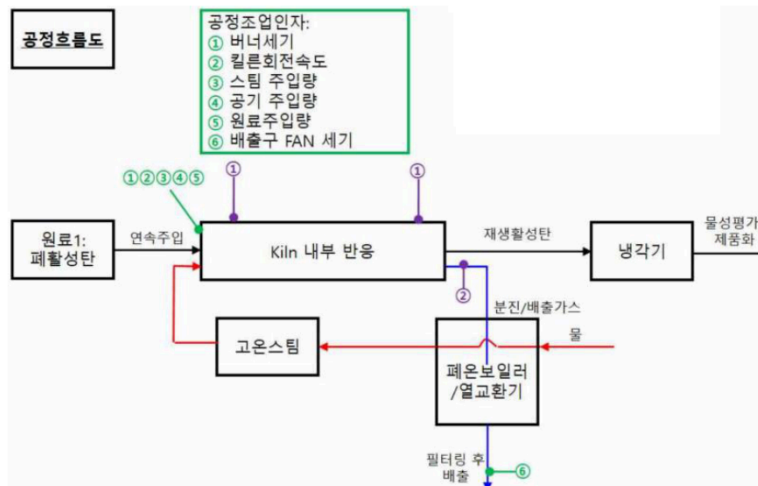
[2강] 인공지능망 & 딥러닝(Deep Learning) 개요

왜 필요한가 — 배경 예제 3가지

1. **활성탄 재생 공정(1강에서도 언급된 그 회사)**: 이번엔 훨씬 많은 공정변수가 등장 — 버너 세기, 킬른 회전속도, 스팀 주입량, 공기 주입량, 원료(폐활성탄) 주입 속도, 배출가스 빼내는 세기 등. 이런 다양한 공정조업인자 조합에서 최적 성능을 찾는 건 대부분 조업자의 경험과 직관에 의존하고 있는데, 이걸 데이터 기반으로 예측하는 데 딥러닝이 유용하다는 예고.

📷 [2강 PDF 1페이지 — 활성탄 재생공정 전체 공정흐름도(원료→Kiln 내부반응→냉각기→분진/배출가스→폐온보일러/열교환기), 버너세기/회전속도/스팀주입량 등 공정조업인자 라벨링]

제조데이터와 예측



상당수 제조 공정은 조업자의 경험과 직관으로 최적화

Q. 어떤 공정조업인자 조합에서 최적의 제품성능? 수치형 제조데이터 활용법은?

1. **한전 가스터빈 시스템(예측+분류 동시 예제)**: 고온에서 작동하고 한번 멈추면 손실이 큰 설비라 온도·압력·진동 데이터를 183개 채널에서 계속 시간에 따라 추적함. 데이터가 워낙 많아서 어떤 채널(들)이 중요한지, 여러 채널을 종합해서 이상을 미리 판단해 유지보수 하는 게 머신러닝 없이는 경험에만 의존할 수밖에 없음. 이걸 활성탄 예제(고정된 숫자 조합)와 달리 **시간에 따라 183개 채널이 계속 변하는 시계열 데이터**라는 점이 다름.
2. **이미지 데이터(양품/불량 카메라 판정)**: 공정 마지막 단계에서 카메라로 계속 사진을 찍어서 불량 여부/불량 종류를 판단하는 경우 — 수치형이 아닌 이미지형 데이터에서도 딥러닝이 유용하게 활용 가능.

📄 (PDF 전용 추가 예시) 이미지 제조데이터 슬라이드에 Match(패턴/모양/방향 일치 확인), Area(부품 유무 및 양품 확인), Blemish(큰 대비가 감지되는 영역 확인), Sort(부품 패턴을 인식하고 분류) 4가지 아이콘 예시가 나옴 (출처: fntc.co.kr).

📷 [2강 PDF 2페이지 — 한전 가스터빈 시계열 이상탐지 대시보드 그림(출처: 한전전력연구원), 그리고 Match/Area/Blemish/Sort 4종 아이콘]

제조데이터와 예측/분류

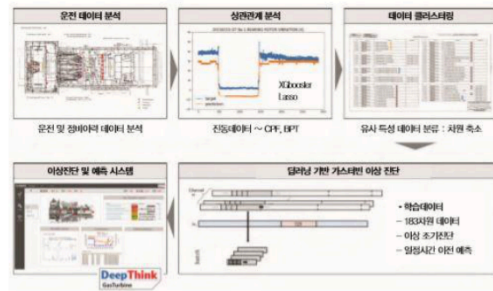


그림 3: 딥러닝 기반 가스터빈 이상진단 및 예측 개요도

<출처: 한전전력연구원, <http://www.keaj.kr/news/articleView.html?idxno=2789>>

한 번 정지하면 큰 손실이 오는 연속공정에서 이상 진단과 예측

Q. 수많은 시계열 데이터 기반으로 이상을 진단하는 방법? 시계열 제조데이터 활용법은?

제조데이터와 분류



Match 패턴,모양,방향 일치 확인

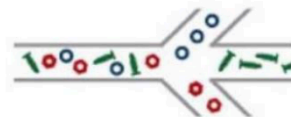


Area 부품 유무 및 양품 확인

<출처: http://fntc.co.kr/app_FV/3334>



Blemish 큰 대비차가 감지되는 영역 확인



Sort 부품의 패턴을 인식하고 분류

제조데이터는 수치 형태 뿐만 아니라 이미지 형태로도 존재

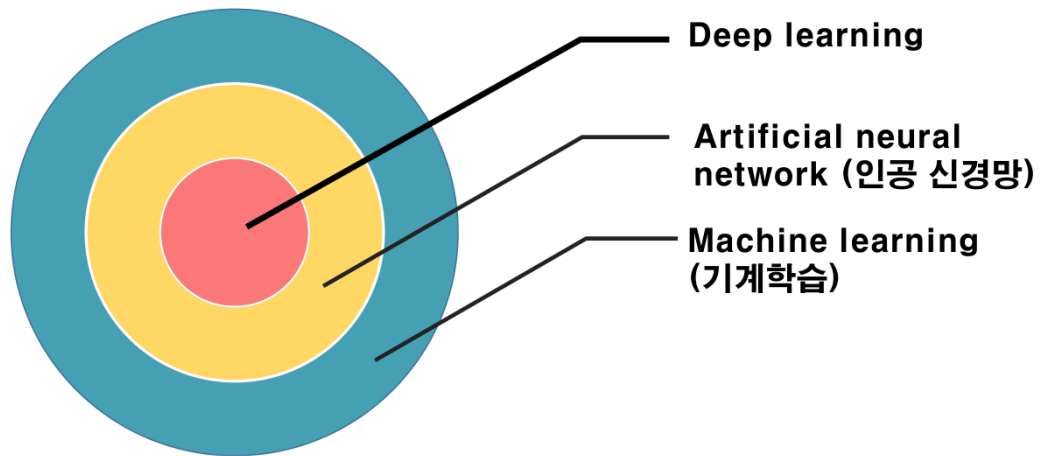
Q. 이미지 제조데이터를 효과적으로 활용하는 방법은?

AI ⊃ ML ⊃ DL 구조 다시 정리

동심원 그림으로 Machine learning ⊃ Artificial neural network(인공신경망) ⊃ Deep learning 관계를 보여줌.

📷 [2강 PDF 3페이지 — AI/ML/DL 3중 동심원 다이어그램]

딥 러닝(Deep learning)



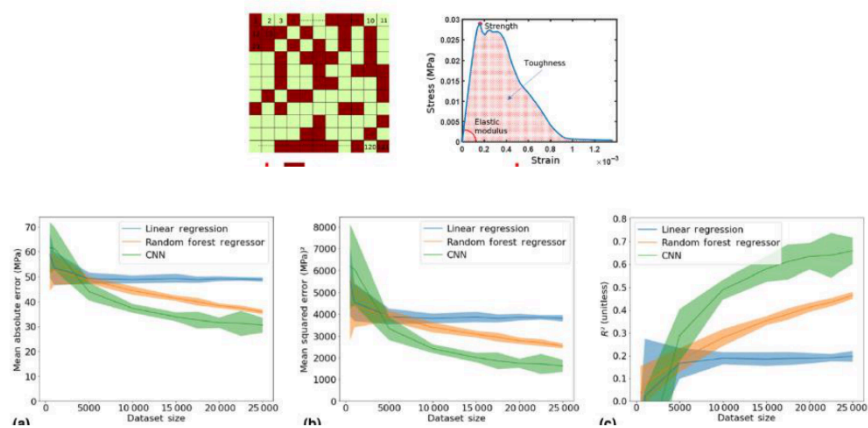
딥러닝의 장점 — 연구실 사례

저자 연구실에서 한 2차원 복합재 필름 연구: 121개 격자에 딱딱한 재료/부드러운 재료 두 종류를 무작위 배치해서 만든 복합재의 응력-변형률 곡선을 예측하는 문제. 격자 121개, 각 칸당 선택지 2개 → **2^{121} 가지 경우의 수**(10의 37승 규모)의 초거대 설계공간. 약 10만 개 학습데이터가 있으면 딥러닝이 응력-변형률 커브를 잘 예측했다고 함.

데이터 크기별 비교 실험: 선형회귀(Linear regression), 랜덤포레스트(Random forest regressor), CNN(딥러닝) 세 모델을 비교했더니 — 데이터셋이 작을 때는 세 모델 성능차가 적고 오히려 인공신경망이 더 안 좋았지만, **데이터가 많아질수록 인공신경망(CNN)이 오차를 가장 잘 줄여나감**. → "데이터가 많으면 많을수록 딥러닝의 성능이 좋아진다"는 딥러닝의 핵심 장점.

📷 [2장 PDF 3페이지 하단 — 121격자 복합재 필름 이미지, 응력-변형률 곡선, 그리고 데이터셋 크기에 따른 MAE/MSE/R² 비교 그래프(선형회귀 vs 랜덤포레스트 vs CNN)]

딥 러닝(Deep learning) 장점



데이터가 많을수록 딥러닝 성능 ↑

인공신경망(퍼셉트론) 개념

사람이 숫자 8을 볼 때, 그 색깔에 해당하는 정보가 시신경을 따라 전파되고 망막에서 전기신호로 바뀌어 뉴런에 전달됨. 이 원리를 컴퓨터로 흉내낸 게 인공신경망.

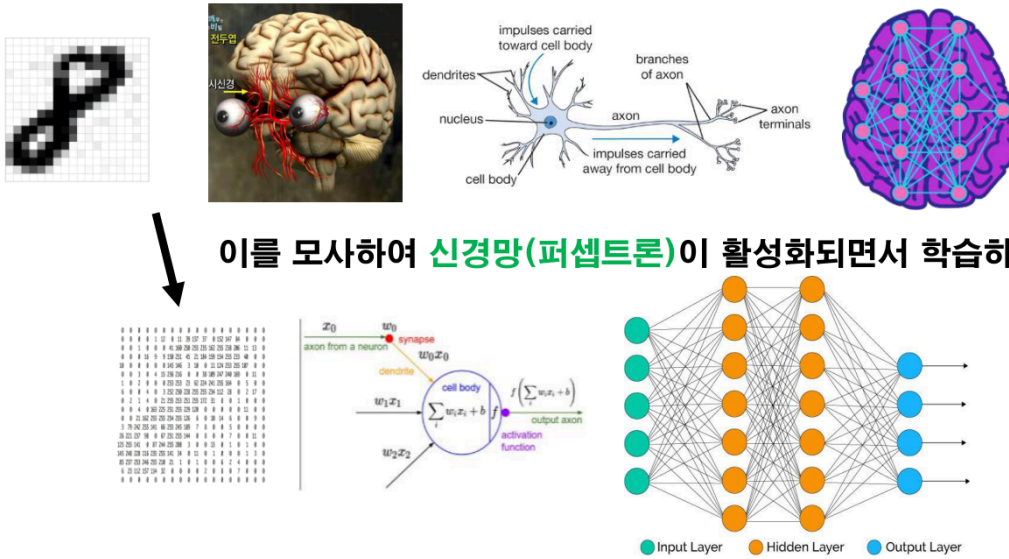
숫자 이미지의 각 픽셀은 검은색이면 255, 흰색이면 0(또는 그 사이 값)으로 숫자화됨. 이 숫자들($x_0, x_1, x_2, \dots$)을 신경망 노드에 입력하고, 각각에 가중치($w_0, w_1, w_2, \dots$)를 곱해서 더하고, 상수(바이어스, b)를 더해서, 거기에 함수 f 를 씌운 결과를 우리가 예측하려는 값과 비교하는 방식으로 학습.

$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

이걸 **퍼셉트론(Perceptron)**이라고 부르고, 이것 여러 개 겹쳐 쌓은 게 **인공신경망**. 사람 뉴런에서 한 뉴런이 여러 뉴런에 연결되고, 여러 경로에서 들어온 전기신호가 합쳐져서 전달되는 구조와 유사하다고 설명.

📷 [2강 PDF 4~5페이지 — 숫자 8의 픽셀 이미지, 사람 뉴런 그림(가지돌기→축삭→다른뉴런), 그리고 퍼셉트론 수식/다이아그램 $x_1, x_2, x_3 \rightarrow \Sigma \rightarrow f \rightarrow y$]

인공 신경망?!



활성함수(activation function)와 손실함수

활성함수 f 가 없으면 이건 그냥 선형회귀랑 똑같은. **비선형 함수(활성함수)**를 넣어줘야 인공신경망이 기존 학습모델보다 유연하게 학습할 수 있음. 대표적으로 **시그모이드**, **렐루(ReLU)** 함수가 있음.

실제 예측값 y 와 $f(w_1x_1 + w_2x_2 + w_3x_3 + b)$ 의 차이 = **손실함수(loss)**. 신경망을 훈련한다는 것은 이 y 값과 f 값의 차이 (=손실함수)를 최소화하는 w 와 b 의 조합을 찾는 것. 기존 선형회귀와 비슷하지만, 선형회귀에는 이 비선형 함수 f 가 없고 단순 계수 조합만 봤다면, 여기서는 비선형 함수까지 포함한 w, b 조합을 찾는다는 차이.

다층 신경망(Multilayer Perceptron)

퍼셉트론을 여러 개 겹쳐서 만든 것. 입력층 - 은닉층(히든레이어) - 출력층 구조. 은닉층은 넓게(노드 수 많게) 쌓을 수도 있고 깊게(층수 많게) 쌓을 수도 있는데, **이건 순전히 사용자의 자유이자 선택 사항**. 데이터가 많으면 파라미터를 많이 쓸 수 있으니 넓고 깊게, 데이터가 적으면 상대적으로 적게 쓰는 게 좋음. **하지만 머신러닝을 전공한 사람이라도 데이터만 보고 은닉층을 몇 층, 얼마나 넓게 써야 하는지는 바로 알 수 없고, 사용자의 경험과 여러 시도로 최적 조합을 찾아야 한다고 강조.**

순전파(Forward Propagation) / 역전파(Back Propagation)

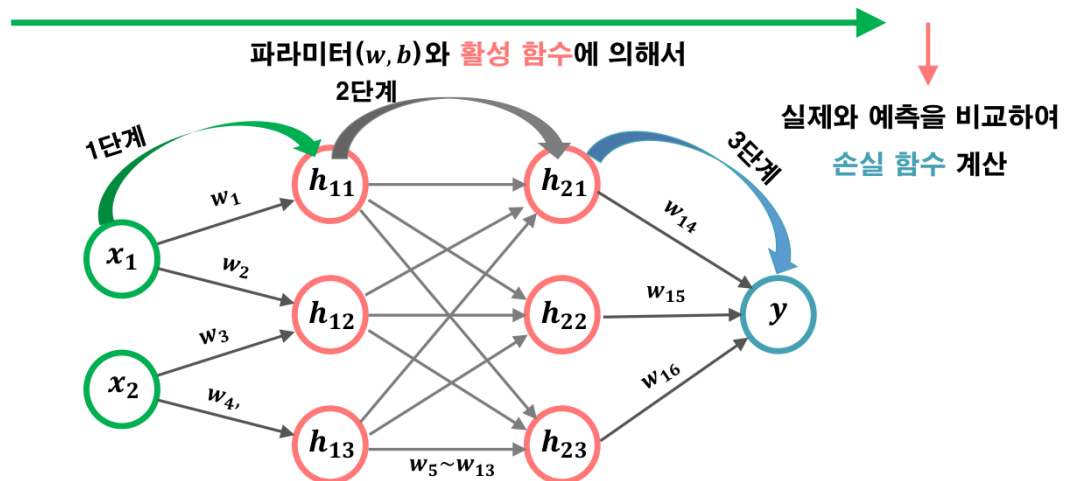
- **순전파**: 입력값($x_1, x_2, \dots$)을 넣으면 각 노드에 가중치를 곱해서 은닉층으로 넘어가고, 각 은닉층 노드에서 이전 층 정보들이 합쳐지고 바이어스가 더해지고 활성화함수를 거쳐 다음 층으로 계속 퍼짐. 최종적으로 예측값 y 가 나옴.
- **역전파**: 예측값과 실제값의 차이(손실함수)를 계산한 뒤, 그 손실함수를 최소화하는 방향으로 **뒤에서부터** 파라미터(w, b)를 개선(조정)해나가는 과정. 이걸 실제로 체계적으로 계산할 수 있게 만든 게 초기 인공신경망 개발자들의 큰 업적이라고 강조. 파라미터가 많아지면 계산량이 많아지기 때문에 **GPU**가 꼭 필요하다고 언급(알파고, 캄프의 복잡한 모델 예시).
- 손실함수를 최소화하는 방법 = **경사하강법(Gradient descent)**.

📷 [2강 PDF 7페이지 — 순전파(1단계 입력→2단계 은닉층 통과→3단계 손실함수 계산)와 역전파(반대 방향으로 파라미터 조정) 흐름 다이어그램]

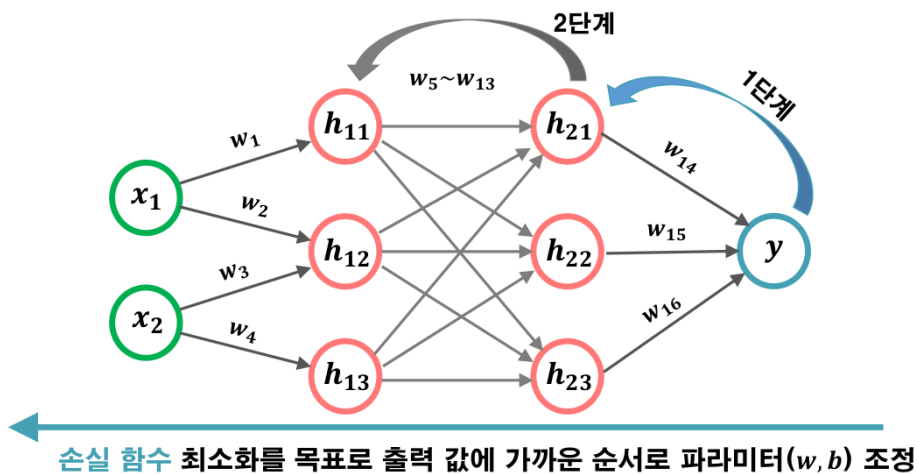
순전파(Forward Propagation)

입력값 $x_1, x_2, \dots$ 을 넣으면

예측되는 출력 y 가 계산



역전파(Back Propagation)



How? → 경사 하강법(Gradient descent): $\frac{\partial Loss}{\partial w}$ 를 계산하여 퍼셉트론 학습

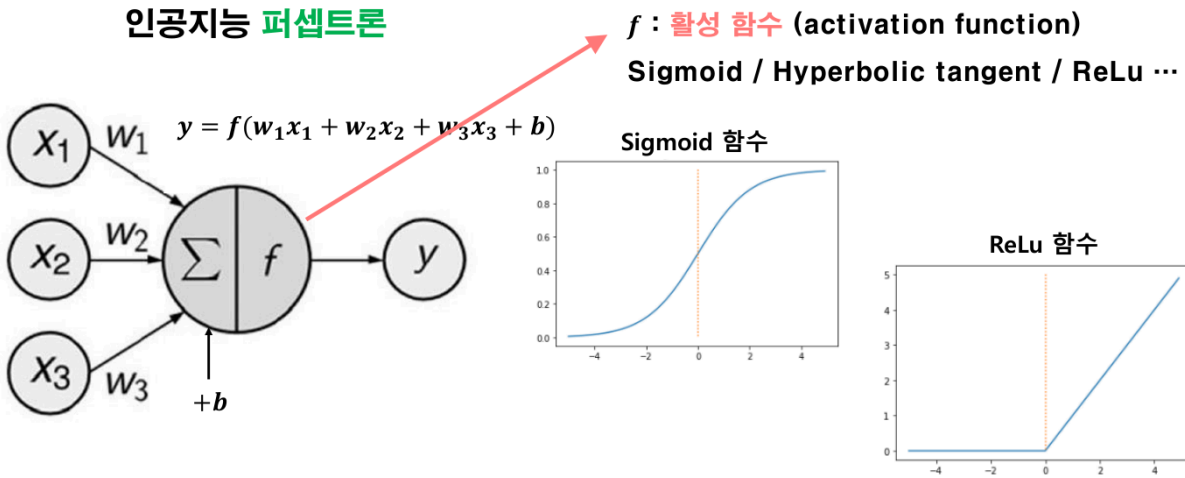
왜 렐루(ReLU)를 시그모이드 대신 많이 쓰는가

은닉층이 두꺼워질수록(층이 많아질수록) 역전파 과정에서 기울기가 잘 전달되지 않는 문제가 있음. 시그모이드 함수는 입력값이 커지면 기울기가 0에 가까워지는 구간이 있어서, 역전파 중 이 기울기가 한 번이라도 곱해지면 그 뒤로는 정보가 다 사라져버림 (그래디언트 소실). 그래서 기울기가 0으로 수렴하지 않는 렐루(또는 리키렐루) 함수를 요즘 더 많이 씀.

📷 [2장 PDF 8페이지 — 시그모이드(기울기 0으로 수렴하는 부분 표시) → ReLU / Leaky ReLU(기울기 일정하게 유지) 비교 그래프]

신경망이란?

인공지능 퍼셉트론



딥러닝 사용 과정 (4단계)

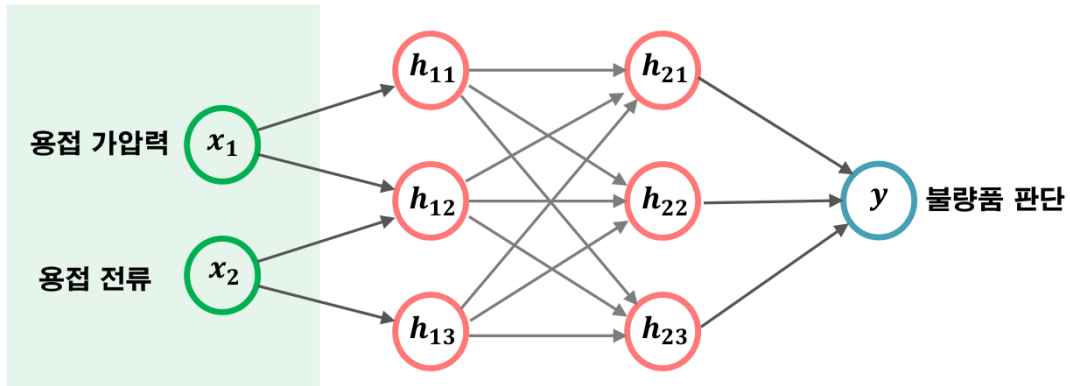
- 적절한 입력 조건 설정:** 사용자의 배경지식이 들어가는 부분. 예: 용접 공정에서 압력·전류가 중요하다는 걸 경험자는 알지만, 온도·습도 등 다른 데이터를 다 넣기보다 필요한 것만 골라 넣는 게 좋음. 단, **입력-출력 관계가 너무 복잡해서 뭐가 중요한지 미리 알 수 없는 경우엔** 오히려 딥러닝의 장점이 발휘됨 — 모든 입력치를 다 넣고 학습시키면, 딥러닝이 알아서 중요한 변수는 높은 가중치로, 안 중요한 변수(예: 용접 성능 예측에 "그날 미국 날씨"를 넣으면)는 은닉층을 거치며 거의 영향이 사라지도록 스스로 학습함.
- 하이퍼파라미터 설정:** 은닉층의 층수/너비 등 신경망 구조 결정. 역시 데이터셋만 보고 바로 정할 수 없어서 여러 조합을 시도 해봐야 함. GPU가 있으면 이런 시도를 짧은 시간에 여러 번 해볼 수 있음.
- 모델 테스트(하이퍼파라미터 튜닝):** 학습데이터로 학습시키고 검증(테스트)데이터로 성능 확인. 데이터를 전부 한꺼번에 넣지 않고 예를 들어 500개씩 랜덤하게 뽑아 넣으면서(배치 학습) 손실함수를 계속 추적함. 학습데이터의 손실함수는 계속 줄어들지만, 검증데이터의 손실함수는 줄어들다가 다시 커지는 지점이 생김.
- 검증데이터 손실함수가 큰 초반 구간 = **과소적합(Underfitting)**
- 검증데이터 손실함수가 줄다가 다시 커지는 구간 = **과대적합(Overfitting)**
- 과소적합 극복법:** 신경망/파라미터 수를 늘리거나 데이터셋을 키우고 학습을 더 많이 진행 → **상대적으로 해결이 쉬움**
- 과대적합 극복법:** 신경망/파라미터 수를 줄이거나 연결관계를 바꿔줌 → **상대적으로 해결이 어렵고, 정해진 방법이 없어서 다양한 시도가 필요함(경험 필요)**

📷 [2장 PDF 8~10페이지 — 용접(가압력 x_1 , 전류 x_2) 입력→은닉층→불량품판단 y 다이어그램(딥러닝 사용과정 1~3단계), 그리고 Underfitting/Overfitting 손실함수 곡선 그래프 2장]

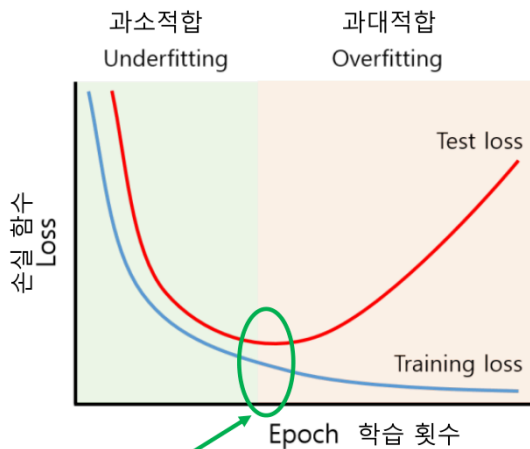
딥러닝 사용 과정

1. 적절한 입력 조건 설정 : 사용자의 배경 지식이 들어가는 부분.

가지고 있는 데이터를 활용 (데이터 전처리 과정 필요)



모델 학습 결과

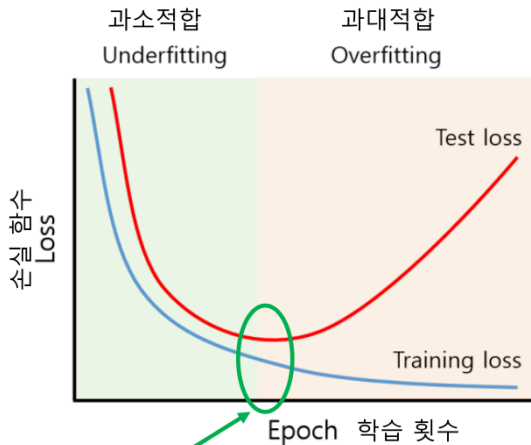


과소적합(Underfitting)을 극복하는 방법

- 신경망과 파라미터 수를 늘린다.
- 학습을 더 많이 진행한다.

→ 상대적으로 해결이 쉽다.

모델 학습 결과



과대적합(Overfitting)을 극복하는 방법

- 신경망과 파라미터 수를 줄인다.
- 신경망의 연결 관계를 바꾸어 준다.

→ 상대적으로 해결이 어렵다.

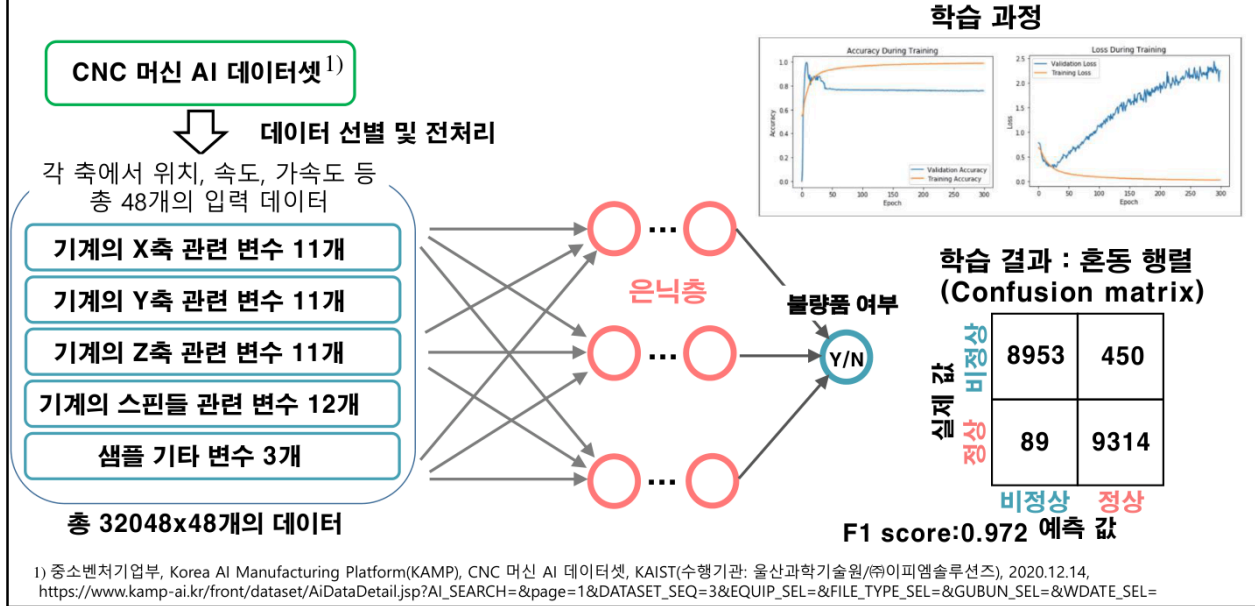
→ 정해진 방법이 없고 다양한 방법 시도.

적용 사례 — CNC 머신 데이터셋 (KAMP)

(PDF 세부수치) 부품의 실제/참조 x,y,z 위치·속도·가속도, x,y,z 전류·전압, 스피들 전류·전압 등 총 **48개 입력변수**. 데이터: 양품 22,645개 + 불량품 9,403개(총 3만2,048개). 딥러닝(심층신경망)에 이 48개를 넣어 학습시킨 결과 — "사람이 이 48개 숫자만 보고 양품/불량 예측하는 건 매우 어렵지만, 심층신경망이 알아서 이 중 중요 요소를 찾아내고 패턴을 분석해서 예측 가능"하다는 점을 강조. Confusion matrix: 8,953 / 450 / 89 / 9,314. F1 score: 0.972 (⚠️ 스크립트에서는 "정상 만개 가까이... 9300개, 450개 예측 틀림... 비정상 예측 9000개 넘음, 89개만 정상" 정도로 설명되는데 PDF 수치와 대체로 일치함)

[2장 PDF 11페이지 — CNC 부품 양품/불량 사진, 48개 변수 입력→은닉층→Y/N 출력 다이어그램, 학습과정 accuracy/loss 그래프, confusion matrix]

딥러닝 적용 사례



딥러닝 활용 3가지 예고 (3~5강 미리보기)

- CNN(합성곱신경망):** 입력값 사이에 관계가 있을 때(예: 이미지 픽셀 간 위치 상관관계) 사용. 3강에서 자세히.
- RNN(순환신경망):** 입력·출력이 시간에 대한 함수로 주어지는 시계열 데이터 학습에 적합. 4강에서 자세히.
- 오토인코더(Autoencoder):** 입력과 출력이 최대한 같도록 학습하는 방법. 예를 들어 입력변수 100개를 5개로 압축(차원 축소)했다가 다시 원래대로 복원하는 대칭 신경망을 만들 수 있으면, 이걸로 차원축소나 새로운 데이터 생성에 활용 가능. 5강에서 설명.

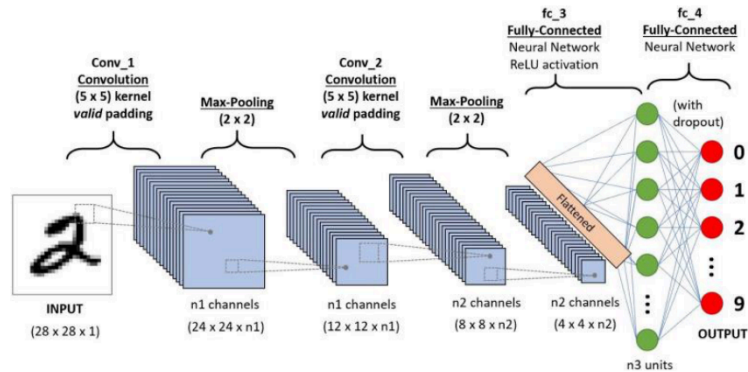
📷 [2강 PDF 12~13페이지 — CNN 구조도(Conv→Pooling→FC), RNN 구조도($x_{<1>}\sim x$ 시퀀스), 오토인코더 구조도(입력 $x_1\sim x_6\rightarrow$ 인코더→잠재공간 $h_1, h_2\rightarrow$ 디코더→출력 $\hat{x}_1\sim\hat{x}_6$)]

딥 러닝 활용 1.CNN

입력 값(Input feature)들 사이의 관계가 있을 때

→ Convolution Neural Network 사용.

→ 특히 이미지 데이터 학습에 적합함.

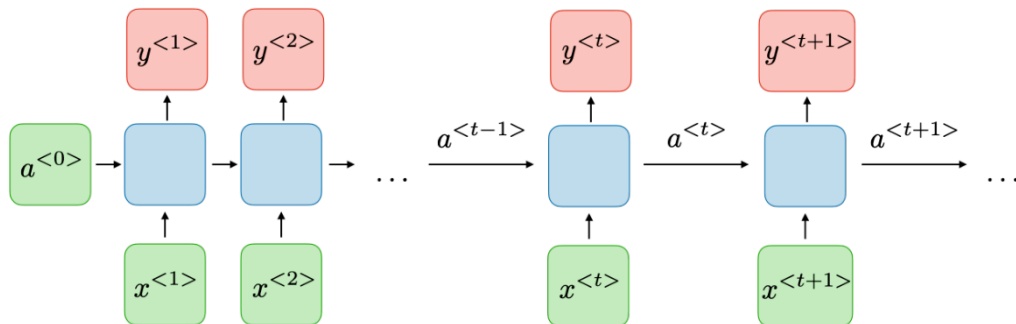


딥 러닝 활용 2.RNN

입력 값과 출력 값이 시퀀스로 이루어진 상황

→ Recurrent Neural Network 사용.

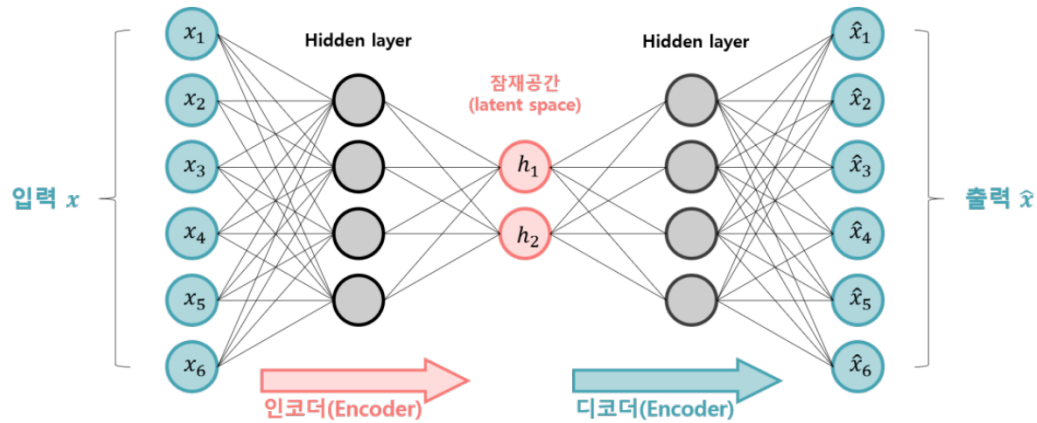
→ 특히 시계열 데이터 학습에 적합함.



딥 러닝 활용 3.Autoencoder

입력과 출력이 최대한 같도록 인공신경망을 학습

→ 차원 축소 및 새로운 데이터 생성 가능



요약

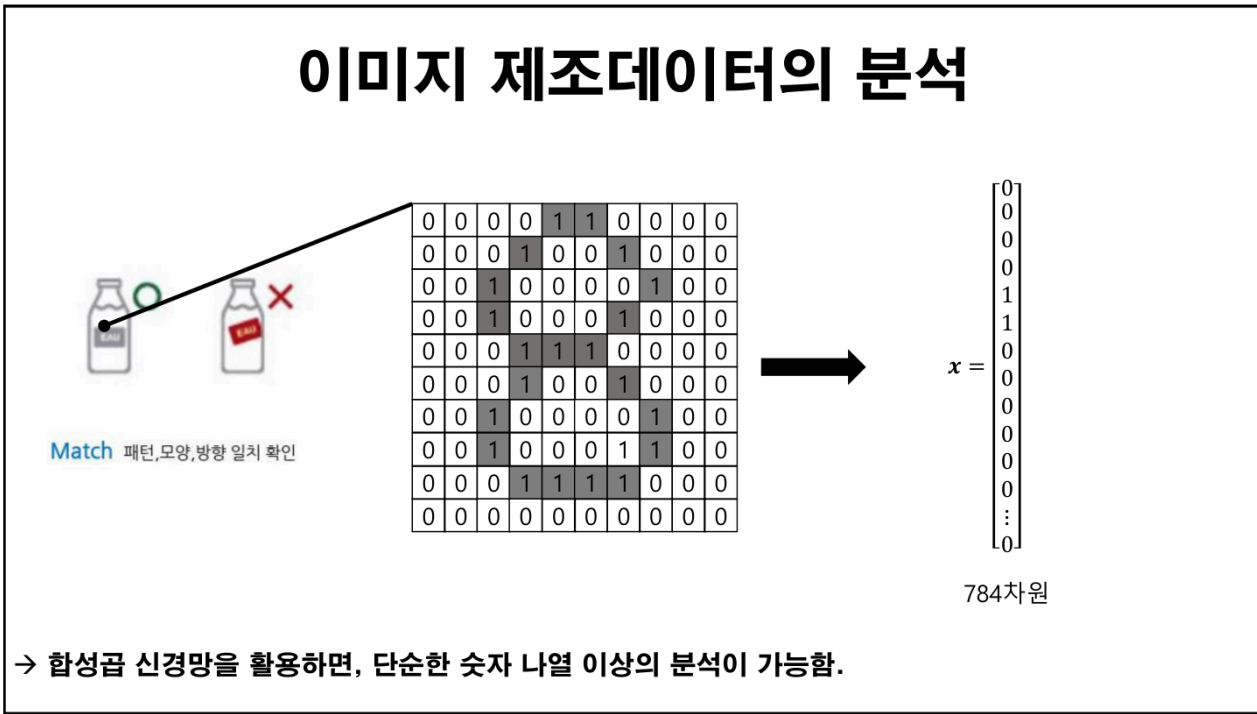
- 퍼셉트론: 인공지능 신경망의 기본 단위(사람 뉴런 모사)
- 여러 층의 퍼셉트론 사용 → 딥러닝
- 딥러닝은 신경망의 활성화 함수와 손실 함수를 정하고, 신경망의 파라미터를 조정하며 모델이 학습됨
- 데이터가 많을수록 우수한 성능을 보임
- 적절한 하이퍼파라미터 튜닝 과정이 필요함

1003

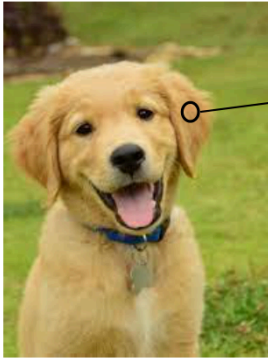
11

ス

C



합성곱 신경망(CNN)



R:198
G:148
B:85



→ 단순히 각 점의 절대값만 아는 것으로는 판별할 수가 없다.

합성곱(Convolution)이란 — 1차원 예시와 주식 이동평균선 비유

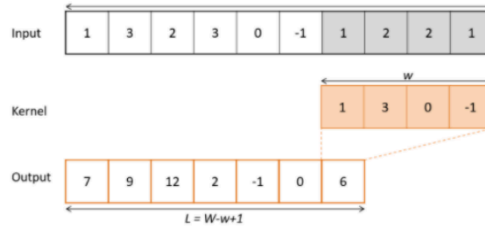
1차원 합성곱 개념: 숫자 나열 1, 3, 2, 3, 0, -1, 1, 2, 2, 1 이 있고, 너비 4짜리 커널(필터) 1, 3, 0, -1 이 있다고 하면, 커널을 앞에서부터 순서대로 각 위치의 숫자와 곱해서 더하는 걸(sliding) 반복해서 출력값(7, 9, 12, 2, -1, 0, 6 등)을 만들. 필터 값이 다 1/n이면 이동평균이 되는 원리.

주식 이동평균선 비유: 매일매일 변하는 주식 가격 데이터에 5일선/20일선/60일선을 씌우는 것 = 각각 너비 5, 20, 60짜리 커널에 모든 칸이 1/5, 1/20, 1/60인 필터를 씌우는 것과 같음. 원래 값을 그냥 보는 것과 다르게, 이 이동평균선들은 "짧은 트렌드/긴 트렌드"에 대한 정보를 알려줌 → 즉 **합성곱이라는 연산을 하면 데이터에서 새로운 직관을 얻을 수 있다**는 걸 이 비유로 설명.

📷 [3강 PDF 3페이지 — 1차원 합성곱 계산 예시(Input/Kernel/Output 표), 그리고 주식 차트에 5일/20일/60일 이동평균선(골든크로스/데드크로스 표시)이 그려진 그림]

합성곱

1차원 합성곱의 개념



1차원 합성곱의 예



Input: 매일의 주식 가격

Kernel → 5일 이동평균: 5칸에 모두 1/5

→ 20일 이동평균: 20칸에 모두 1/20

→ 60일 이동평균: 60칸에 모두 1/60

Output: 5일, 20일, 60일 이동평균

각 Kernel 합성곱 후 단순한 숫자 나열 이상의 정보를 표시

2차원 합성곱 — 이미지 필터 예시

이미지(Lena 사진)에 3x3 필터를 적용하는 예시 2가지:

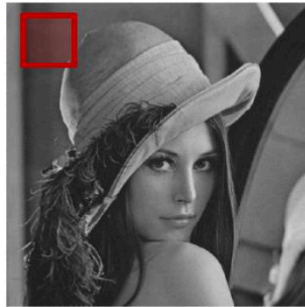
- 필터 $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ (좌우 대비) → 수직 방향 컨트라스트가 강조된 이미지가 나옴(수평선은 잘 안 나옴)

- 필터 $\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$ (상하 대비) → 반대로 수평 방향이 강조되고 수직선은 사라짐

이런 식으로 수직선 강조 필터, 수평선 강조 필터, 대칭/비대칭 강조 필터 등을 여러 개 활용해서 정보를 조합한 다음 딥러닝 입력으로 넣으면 더 효과적으로 배울 수 있지 않을까? → 이게 CNN의 아이디어.

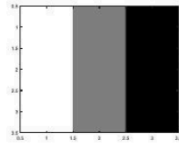
[3장 PDF 4페이지 — Lena 사진에 수직강조 커널, 수평강조 커널을 각각 적용한 결과 이미지 2장(흑백 대비 뚜렷)]

합성곱



Image

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$



Kernel



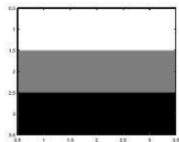
Output

합성곱



Image

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



Kernel

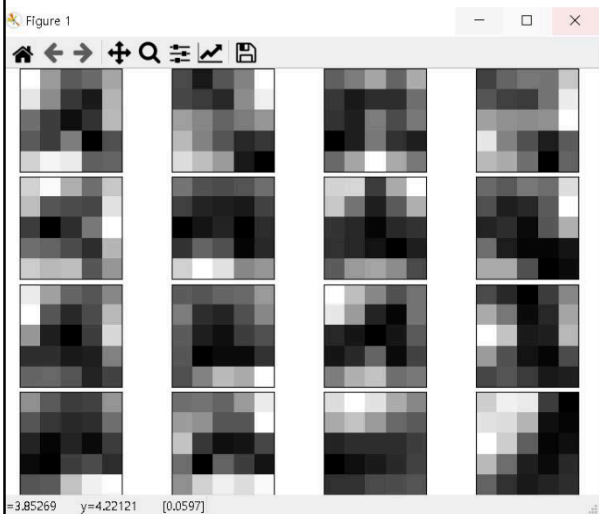


Output

CNN이 학습을 통해 알아서 어떤 필터가 적절한지 찾아냄. 실제로 CNN 학습 후 얻어진 16개의 5x5 필터를 원본 이미지에 적용한 결과를 보여주는 예시도 있는데, 각 필터마다 그림의 다른 패턴/직관이 추출됨.

 [3강 PDF 5페이지 — CNN 학습 후 얻은 16개 커널(왼쪽) 및 각 커널을 원본 이미지에 합성곱한 결과 이미지 16장(우측), 출처: huangdi.tistory.com]

합성곱



CNN으로 학습 후 커널(좌)과 합성곱 처리된 이미지(우)
출처: <https://huangdi.tistory.com/36>

필터를 여러 개(예: 24개) 써서 이미지 하나로 24개의 새로운 이미지(피쳐맵)를 만들고, 그 다음 풀링으로 파라미터 수를 줄이고, 이 과정을 반복하다가 최종적으로 딥러닝에 넣어 0~9 분류 등을 함. → 딥러닝에 넣는 입력 숫자가 원본 RGB 값이 아니라 **필터(커널)**를 거쳐서 나온, 직관을 담은 숫자가 된다는 게 CNN의 핵심.

CNN의 다른 활용 예시 (NEW 이미지 이외)

- 이미지를 R/G/B 3채널로 분리한 뒤 각각 커널 적용
- 재료과학: 화합물의 조성비(구리/알루미늄/산소 비율 등)를 주기율표 이미지 형태로 변환한 다음 CNN으로 물성을 학습
- 저자 연구실 사례: 3D 프린터로 두 물질을 격자형 필름으로 만들 때, 그 물성을 CNN으로 예측($2^{25} \approx 3$ 천만 가지 경우의 수 문제도 논문으로 다룸)

 [3강 PDF 6페이지 — RGB 3채널 분리 이미지, 화합물 조성비→주기율표 이미지 변환 예시, 복합재 패턴(Soft/Hard 격자) 그림]

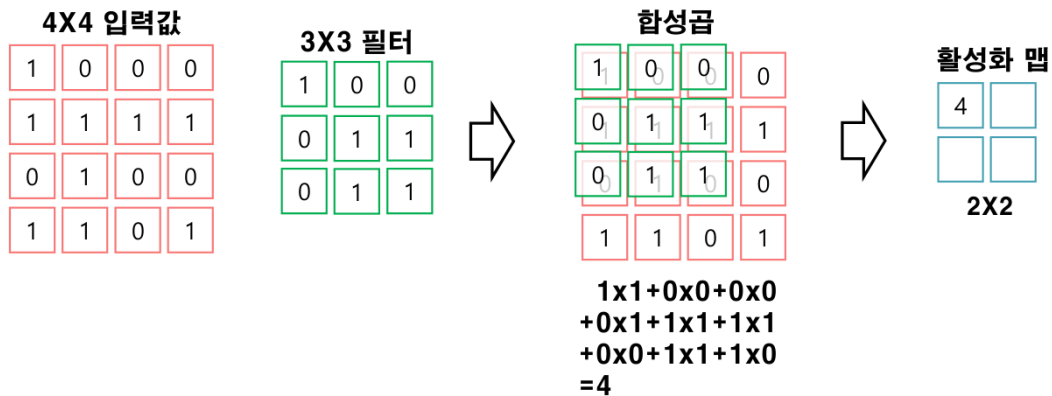
합성곱 신경망 활용 예시



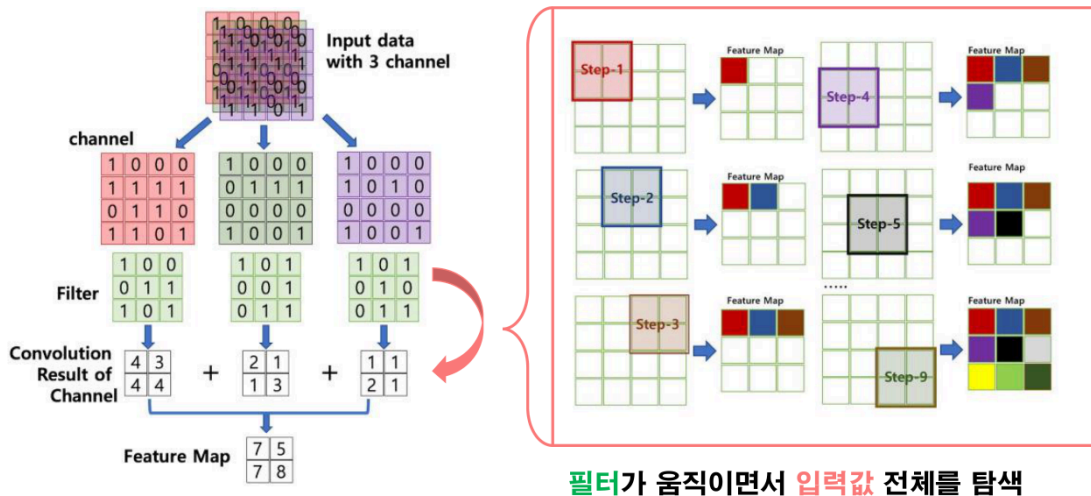
이미지 -> RGB값으로 분리

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	525	526	527	528	529	530	531	532	533	534	535	536	537	538	539	540	541	542	543	544	545	546	547	548	549	550	551	552	553	554	555	556	557	558	559	560	561	562	563	564	565	566	567	568	569	570	571	572	573	574	575	576	577	578	579	580	581	582	583	584	585	586	587	588	589	590	591	592	593	594	595	596	597	598	599	600	601	602	603	604	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620	621	622	623	624	625	626	627	628	629	630	631	632	633	634	635	636	637	638	639	640	641	642	643	644	645	646	647	648	649	650	651	652	653	654	655	656	657	658	659	660	661	662	663	664	665	666	667	668	669	670	671	672	673	674	675	676	677	678	679	680	681	682	683	684	685	686	687	688	689	690	691	692	693	694	695	696	697	698	699	700	701	702	703	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720	721	722	723	724	725	726	727	728	729	730	731	732	733	734	735	736	737	738	739	740	741	742	743	744	745	746	747	748	749	750	751	752	753	754	755	756	757	758	759	760	761	762	763	764	765	766	767	768	769	770	771	772	773	774	775	776	777	778	779	780	781	782	783	784	785	786	787	788	789	790	791	792	793	794	795	796	797	798	799	800	801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820	821	822	823	824	825	826	827	828	829	830	831	832	833	834	835	836	837	838	839	840	841	842	843	844	845	846	847	848	849	850	851	852	853	854	855	856	857	858	859	860	861	862	863	864	865	866	867	868	869	870	871	872	873	874	875	876	877	878	879	880	881	882	883	884	885	886	887	888	889	890	891	892	893	894	895	896	897	898	899	900	901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920	921	922	923	924	925	926	927	928	929	930	931	932	933	934	935	936	937	938	939	940	941	942	943	944	945	946	947	948	949	950	951	952	953	954	955	956	957	958	959	960	961	962	963	964	965	966	967	968	969	970	971	972	973	974	975	976	977	978	979	980	981	982	983	984	985	986	987	988	989	990	991	992	993	994	995	996	997	998	999	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	1011	1012	1013	1014	1015	1016	1017	1018	1019	1020	1021	1022	1023	1024	1025	1026	1027	1028	1029	1030	1031	1032	1033	1034	1035	1036	1037	1038	1039	1040	1041	1042	1043	1044	1045	1046	1047	1048	1049	1050	1051	1052	1053	1054	1055	1056	1057	1058	1059	1060	1061	1062	1063	1064	1065	1066	1067	1068	1069	1070	1071	1072	1073	1074	1075	1076	1077	1078	1079	1080	1081	1082	1083	1084	1085	1086	1087	1088	1089	1090	1091	1092	1093	1094	1095	1096	1097	1098	1099	1100	1101	1102	1103	1104	1105	1106	1107	1108	1109	1110	1111	1112	1113	1114	1115	1116	1117	1118	1119	1120	1121	1122	1123	1124	1125	1126	1127	1128	1129	1130	1131	1132	1133	1134	1135	1136	1137	1138	1139	1140	1141	1142	1143	1144	1145	1146	1147	1148	1149	1150	1151	1152	1153	1154	1155	1156	1157	1158	1159	1160	1161	1162	1163	1164	1165	1166	1167	1168	1169	1170	1171	1172	1173	1174	1175	1176	1177	1178	1179	1180	1181	1182	1183	1184	1185	1186	1187	1188	1189	1190	1191	1192	1193	1194	1195	1196	1197	1198	1199	1200	1201	1202	1203	1204	1205	1206	1207	1208	1209	1210	1211	1212	1213	1214	1215	1216	1217	1218	1219	1220	1221	1222	1223	1224	1225	1226	1227	1228	1229	1230	1231	1232	1233	1234	1235	1236	1237	1238	1239	1240	1241	1242	1243	1244	1245	1246	1247	1248	1249	1250	1251	1252	1253	1254	1255	1256	1257	1258	1259	1260	1261	1262	1263	1264	1265	1266	1267	1268	1269	1270	1271	1272	1273	1274	1275	1276	1277	1278	1279	1280	1281	1282	1283	1284	1285	1286	1287	1288	1289	1290	1291	1292	1293	1294	1295	1296	1297	1298	1299	1300	1301	1302	1303	1304	1305	1306	1307	1308	1309	1310	1311	1312	1313	1314	1315	1316	1317	1318	1319	1320	1321	1322	1323	1324	1325	1326	1327	1328	1329	1330	1331	1332	1333	1334	1335	1336	1337	1338	1339	1340	1341	1342	1343	1344	1345	1346	1347	1348	1349	1350	1351	1352	1353	1354	1355	1356	1357	1358	1359	1360	1361	1362	1363	1364	1365	1366	1367	1368	1369	1370	1371	1372	1373	1374	1375	1376	1377	1378	1379	1380	1381	1382	1383	1384	1385	1386	1387	1388	1389	1390	1391	1392	1393	1394	1395	1396	1397	1398	1399	1400	1401	1402	1403	1404	1405	1406	1407	1408	1409	1410	1411	1412	1413	1414	1415	1416	1417	1418	1419	1420	1421	1422	1423	1424	1425	1426	1427	1428	1429	1430	1431	1432	1433	1434	1435	1436	1437	1438	1439	1440	1441	1442	1443	1444	1445	1446	1447	1448	1449	1450	1451	1452	1453	1454	1455	1456	1457	1458	1459	1460	1461	1462	1463	1464	1465	1466	1467	1468	1469	1470	1471	1472	1473	1474	1475	1476	1477	1478
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------

합성곱(convolution) 적용



합성곱(convolution) 적용

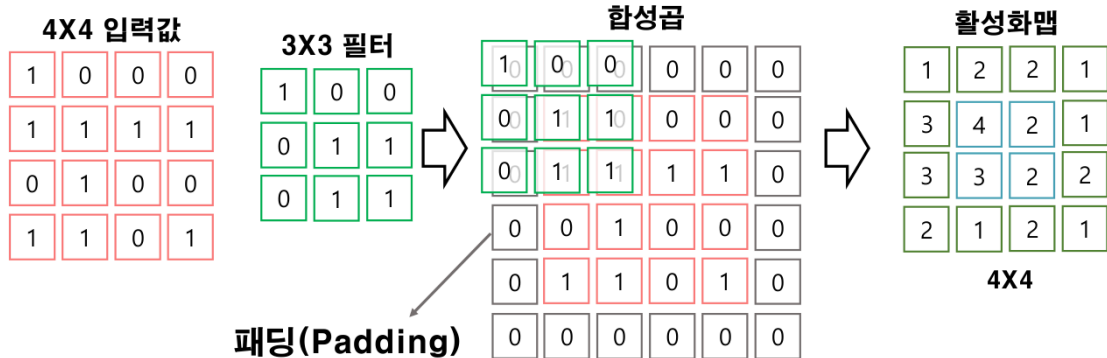


패딩(Padding): 합성곱을 여러 층 반복하고 싶는데 이미지가 계속 작아지면 결국 필터를 씌울 수 없는 크기가 됨. 이것 막기 위해 원본 이미지 테두리에 추가 데이터(보통 0, "제로 패딩")를 붙여서 합성곱 후에도 원래 이미지 크기가 유지되게 하는 방법. 이러면 합성곱을 여러 층에 걸쳐 반복할 수 있음.

풀링(Pooling): 파라미터가 너무 많아지면 과적합 위험이 있으니, 일정 구간에서 가장 큰 값(최대값 풀링)이나 평균값(평균값 풀링)만 뽑아서 **파라미터 수를 줄이는** 레이어. 패딩과 반대 방향의 작업(크기를 줄임).

플래튼(Flatten): 2차원(또는 3차원) 데이터 구조를 1차원으로 펼쳐주는 것. 플래튼 이후는 기존 다층신경망(딥러닝)과 동일하게 손실함수 계산 가능.

패딩(Padding)

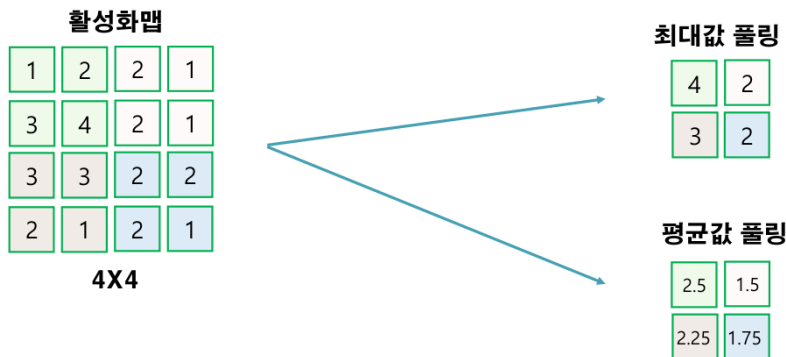


→ 테두리에 **추가 데이터**를 붙인다.

→ 0을 붙이면 제로 패딩(zero padding).

→ 활성화맵의 사이즈가 줄어들지 않아서 **합성곱** 반복 가능

풀링(Pooling)



→ 일정 구간에서 **최대값**이나 **평균값**을 추출.

→ 패딩과 반대로 **파라미터 개수**를 줄여서
과적합(over-fitting)을 피하게 해줌.

CNN 적용 사례 (KAMP 및 자체 연구)

NEW (PDF 세부수치)

- **사례 1 — 제조 현장용 OCR (디지털 저울 아닌 아날로그 저울 숫자 인식)**: 카메라로 찍은 저울 숫자(예: "1720")를 사람은 바로 읽지만, 컴퓨터가 엑셀에 자동 입력하려면 숫자 영역만 골라내고 각 숫자(0~9)를 분류하는 학습이 필요. 데이터 27,237개, **약 98% 정확도**로 인식 성공. (강사 코멘트: "이걸 원래 디지털 저울로 바꾸는 게 가장 좋을 수도 있지만, 그러지 못하는 경우도 있다"는 실용적 코멘트)

- **NEW** KAIST에서 진행한 다른 사례: 매일 파도 사진을 찍고 그날 일기예보의 파고(파도 높이)와 매칭해서 학습시키면, 나중엔 일기 예보 없이 사진만 찍어도 파고를 예측해서 "오늘 조업 나갈 수 있는지" 판단 가능했다는 사례(정확한 수치는 스크립트에 안 나옴,

사진 자체도 첨부 안 됨)

- **사례 2 — Ford 엔진 정상/비정상 (시계열에도 CNN 적용 가능)**: 500개 센서에서 T0~T3600까지 측정한 시계열 데이터를 CNN으로 정상/비정상 분류. "센서들이 완전히 독립적이지 않고 조금씩 관계가 있으면 CNN이 더 잘 예측한다"는 포인트. 데이터 4,921개(학습3,601/테스트1,320). Confusion matrix: 665/16/23/616. F1 score: 0.97

- **사례 3 — 복합재 패턴(3D프린팅 격자 필름)**: Soft/Hard 두 재료의 격자 패턴 이미지를 CNN으로 학습해서 강성·강도 등 물성을 예측(2강에서 언급된 그 연구, 여기서 다시 CNN 관점으로 재소개).

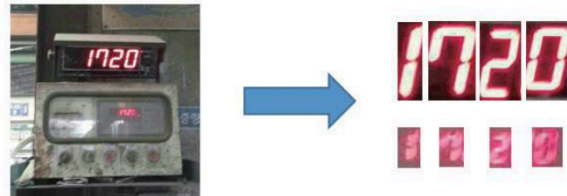
📷 [3강 PDF 10~12페이지 — 저울 숫자 인식 OCR 사례(디지털 숫자 이미지→1,0,1,0 이진화→CNN→플래튼→분류 "2"), Ford 엔진 센서 시계열 그래프+confusion matrix, 복합재 패턴 격자 그림]

CNN 분류 적용 사례 1

CNN 실제 적용 사례 ²⁾

- 목표: 제조 현장용 OCR학습 제조AI 데이터셋
- 인풋: 무게 계량 **저울**을 촬영하여 수집된 **이미지 데이터**
- 아웃풋: **숫자 분류**
- 데이터 수: 27,237 개

데이터 예시



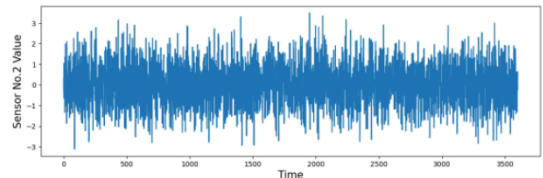
2) 중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 제조 현장용 OCR학습 제조AI 데이터셋, KAIST(산업 및 시스템공학과 문일철 교수님), 2020.12.14, https://www.kamp-ai.kr/front/dataset/AiDataDetail.jsp?AI_SEARCH=&page=1&DATASET_SEQ=1&EQUIP_SEL=FILE_TYPE_SEL=&GUBUN_SEL=&WDATE_SEL=

CNN 분류 적용 사례 2

CNN 실제 적용 사례 ³⁾

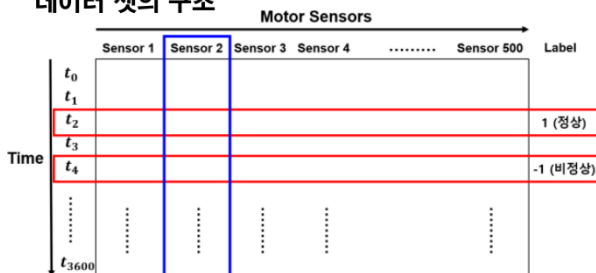
- 목표: Ford 엔진 **정상/비정상** 여부를 센서의 **계측 값**을 통해 예측
- 인풋 (x_t): 500개의 **센서**에서 수집된 **계측 데이터**
- 아웃풋 (h_t): **정상/비정상** 판단
- 데이터 수: 4,921 개 (학습:3,601, 테스트: 1,320)

센서 데이터 예시



-> 이미지 뿐 아니라 **시계열**에도 CNN 적용 가능
혼동 행렬(Confusion matrix)

데이터 셋의 구조

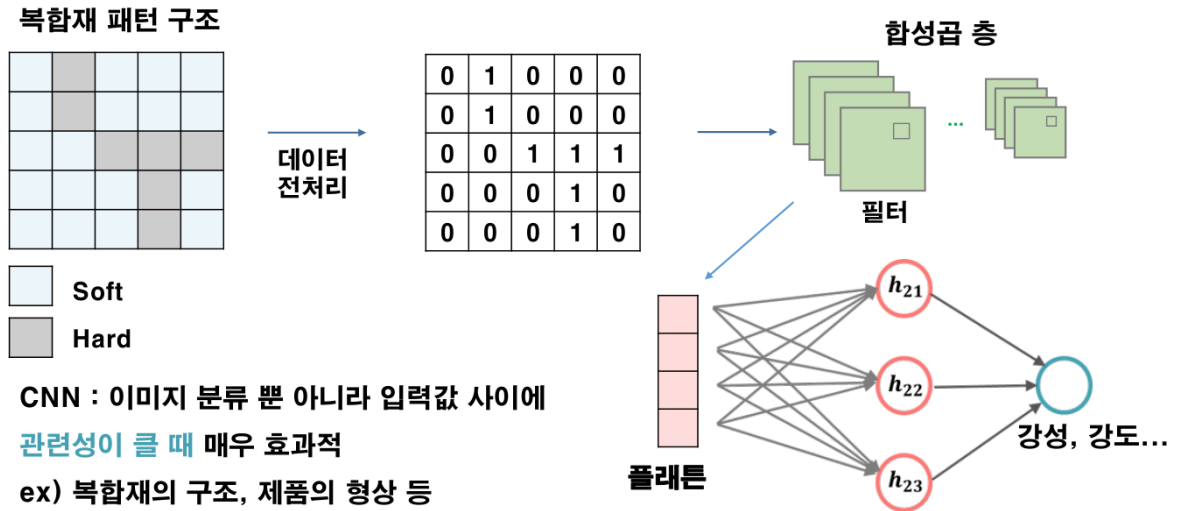


실제 값	정상	665	16
	비정상	23	616
	예측 값	정상	비정상

F1 score:0.97

3) 중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 제조 현장용 OCR학습 제조AI 데이터셋, KAIST(수행기관: ㈜ 임팩스/한양대학교 산학협력단/㈜아큐라스프트), 2020.12.14. https://www.kamp-ai.kr/front/dataset/AiDataDetail.jsp?AI_SEARCH=&name=1&DATASET_SEQ=12&EQUIP_SEL=FILE_TYPE_SEL=&GUBUN_SEL=&WDATE_SEL=

CNN 분류 적용 사례 3



Yongtae Kim, Youngsoo Kim, Charles Yang, Kundo Park, Grace X. Gu, and Seunghwa Ryu, "Deep Learning Framework for Material Design Space Exploration using Active Transfer Learning and Data Augmentation", npj Computational Materials (2021)

요약

- 데이터와 필터의 합성곱을 이용해서 입력값 사이의 패턴을 학습
- CNN은 분류/회귀를 가장 잘하는 커널(필터)을 학습하는 방법
- 주변과의 관계가 중요한 이미지 학습에 탁월하지만, 이미지가 아니라도 옆 데이터끼리 연관성이 있을 때 유용
- 합성곱 외에도 패딩/풀링/플래튼 설정으로 효과적으로 학습
- 필터가 뭐가 적절한지 여러 번 시도하며 학습한 다음, 의미있는 숫자로 바뀌어서 심층신경망에 넣었을 때가 그냥 픽셀값을 직접 넣는 것보다 성능이 확실히 더 좋음

[4강] RNN(순환신경망, Recurrent Neural Network) & LSTM

도입 — 시계열 데이터의 두 가지 접근

엔진 진동 데이터처럼 시간에 대한 함수로 나타난, 불규칙해 보이는 시계열 데이터가 있을 때:

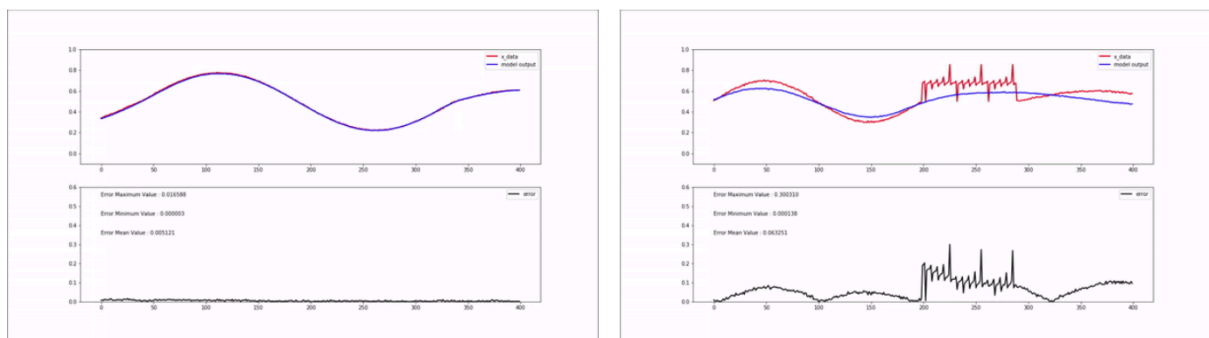
1. **예측 관점:** 사람 직관 없이도 미래에 어떤 데이터가 나올지(패턴) 예측
2. **분류 관점:** 현재 엔진이 정상/비정상인지 알고 있을 때 그 패턴 데이터를 계속 모아서, 얼핏 랜덤한 노이즈처럼 보이는 데이터에서 정상/비정상을 판단

두 기능을 합쳐서: 정상 패턴 데이터를 넣으면 예측값(파란색)이 실제값을 잘 따라가서 에러가 없으면 "정상"으로 판단, 학습에 전혀 쓰이지 않은 새로운(비정상) 데이터를 넣으면 예측값이 엉뚱해지고 에러가 커지므로 "비정상"으로 판단하는 실시간 이상탐지도 가능.

이렇게 **순서(시간 순서)**가 **중요한 데이터**에는 기존 딥러닝(완전연결)이나 CNN보다 **RNN**이 효율적. 기존 신경망/CNN은 순차 데이터가 주어져도 "어느 게 더 먼저인지" 순서를 고려하지 않고 독립적으로 처리하는 한계가 있음.

📷 [4강 PDF 2페이지 — 시계열 데이터에 LSTM-오토인코더로 이상탐지하는 예시 그래프 2장(정상 패턴은 예측선이 잘 따라가고 에러 작음, 비정상은 에러 급증), 출처: inspaceai.github.io]

시계열 제조데이터의 분석



<출처: https://inspaceai.github.io/2019/03/21/Anomaly_Detection_With_LSTM-AE/>

→ RNN(순환 신경망)을 활용하면, 순서가 중요한 순차 데이터를 효율적으로 분석할 수 있음

순차 데이터 / 시계열 데이터 정의: 순차 데이터 = 샘플에 순서가 있는 데이터(ex. 시계열 데이터). 시계열 데이터 = 일정 시간 간격으로 배치된 데이터(기계 장비 센서 데이터, 컴퓨터 시뮬레이션에서 시간마다 달라지는 입력값 등).

RNN 목표 예시 — 사출성형 수지 유동

예제: 사출성형(Injection molding) 시 수지 주입속도 변화에 따른 수지 유동 선단(flow front) 온도를 예측하고 싶은 상황.

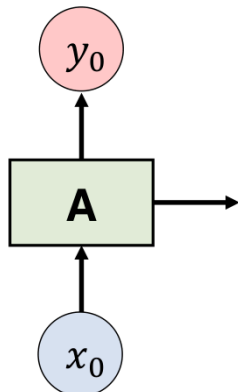
- 인풋: 지속적으로 변하는 수지 주입 속도
- 아웃풋: 수지 유동 선단 온도(또는 속도)

인풋이 $x_0, x_1, x_2 \dots (T_0, T_1, T_2 \text{ 시점마다})$ 로 시간에 따라 바뀌면 아웃풋도 계속 바뀜. **전 단계 정보와 현재 정보가 합쳐져서 새 아웃풋에 영향을 줌** — 실제 물리현상도 현재 주입속도뿐 아니라 "이전에 얼마나 빨리 밀어넣었는지"가 현재 온도에 영향을 미치기 때문. 이것을 인공신경망으로 구현하려는 시도가 RNN.

📷 [4강 PDF 3~5페이지 — RNN 기본 구조: $y_0 \leftarrow A \leftarrow x_0 \rightarrow y_0, y_1 \leftarrow A, A \leftarrow x_0, x_1 \rightarrow \dots y_0 \sim y_3 \leftarrow A, A, A, A \leftarrow x_0 \sim x_3$ 순으로 셀(A)이 늘어나는 4단계 다이어그램, 그리고 h (은닉상태)와 가중치 w_{hh}, w_{xh} 가 표시된 RNN 내부구조 다이어그램]

RNN(순환 신경망)이란?

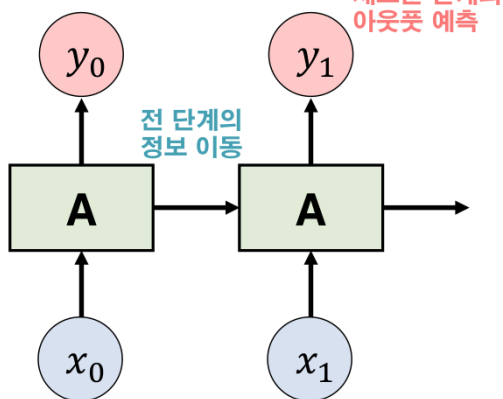
아웃풋 (수지 유동 선단 온도)



인풋 (주입 속도)

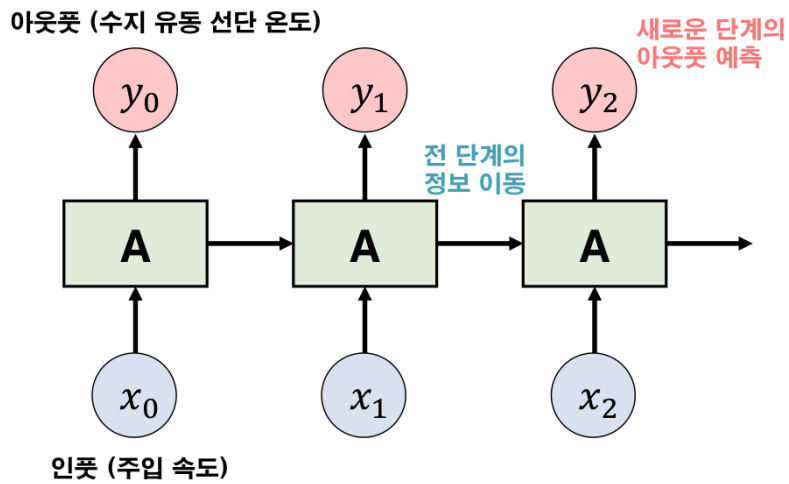
RNN(순환 신경망)이란?

아웃풋 (수지 유동 선단 온도)

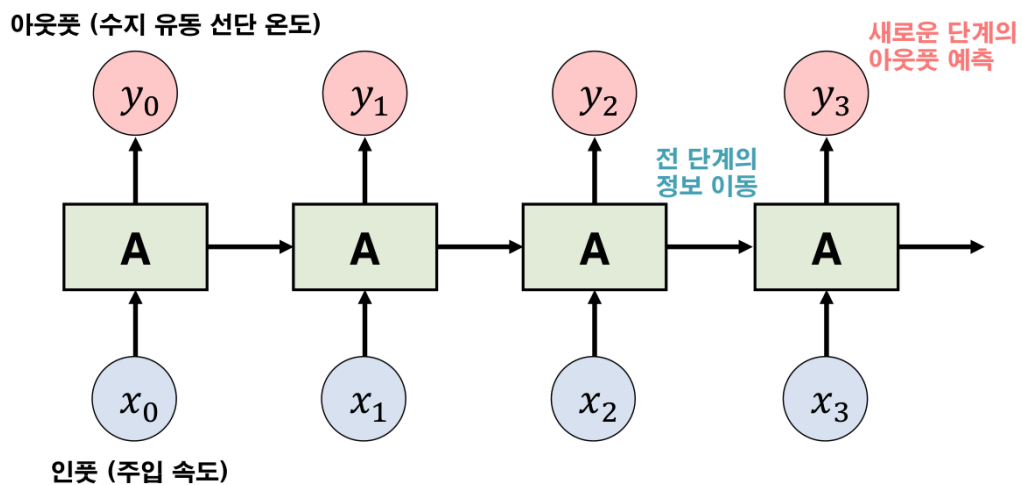


인풋 (주입 속도)

RNN(순환 신경망)이란?



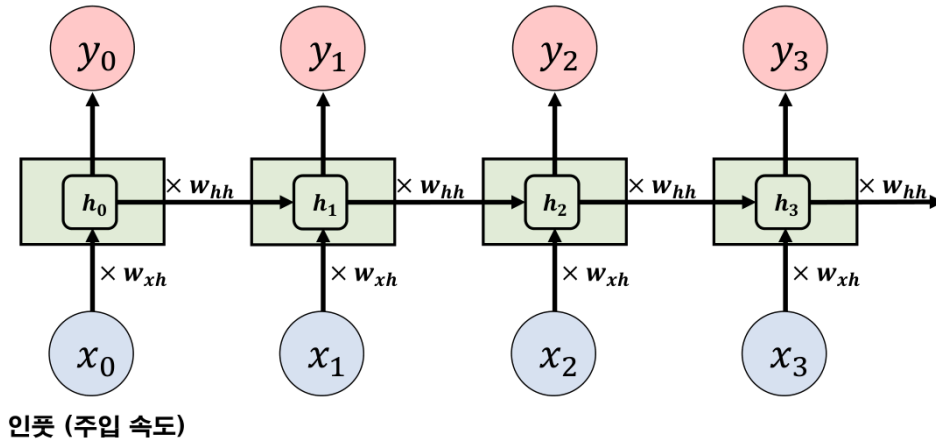
RNN(순환 신경망)이란?



RNN(순환 신경망)이란?

RNN 내부 구조 예시

아웃풋 (수지 유동 선단 온도)



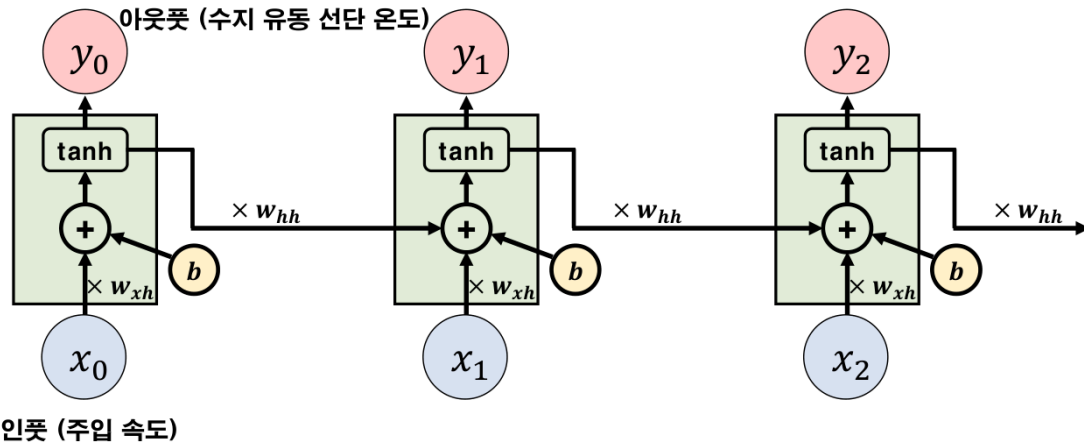
RNN 구조 핵심

- 인풋이 은닉층으로 넘어갈 때, 그리고 현재 은닉층이 다음 스텝 은닉층으로 넘어갈 때 **가중치(w)**가 **완전히 동일함** — 이게 핵심 설계 포인트. 데이터 길이(10초든 20초든)에 상관없이 동일 구조의 가중치를 반복 사용할 수 있게 만든 "순환적" 구조.
- 단순 상수 곱셈만 하면 선형회귀와 다를 게 없으니, **비선형 함수(tanh, -1~1 범위)**를 활성화함수로 씀.
- 인풋은 하나의 변수(주입속도)만이 아니라 여러 개(주입속도+몰드온도+냉각수온도 등)를 벡터로 넣을 수 있고, 아웃풋도 벡터로 여러 개 나갈 수 있음.
- 자연어(문장) 처리도 순서가 중요한 데이터의 예로 언급됨(한국어/영어 어순이 다르다는 코멘트).
- 추가 활용: 시계열 인풋을 다 넣은 후, 유동선단 속도뿐 아니라 최종적으로 "공차 기준 합격/불합격" 같은 최종 판단까지 내리도록 확장 가능(Softmax 레이어 추가).

📷 [4강 PDF 6~7페이지 — tanh 활성화함수를 포함한 RNN 셀 내부구조, 그리고 RNN 마지막에 Softmax를 붙여 최종 합격/불합격 y를 출력하는 구조]

RNN(순환 신경망)이란?

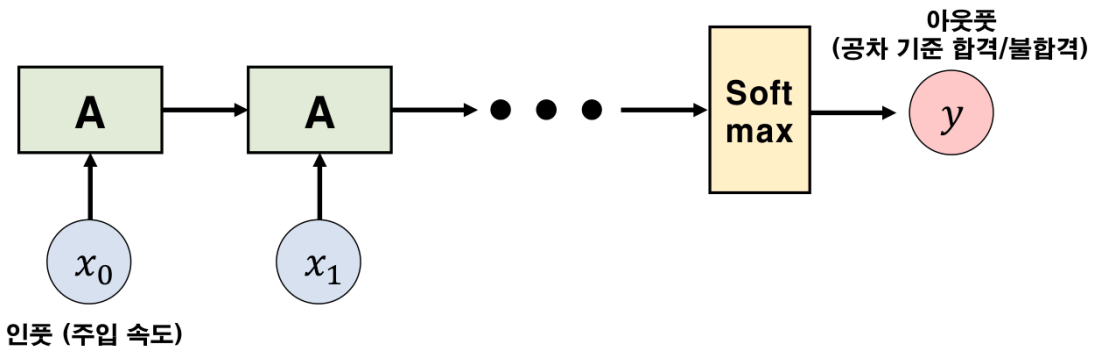
활성 함수(activation function)를 포함한 RNN 구조 예시



RNN(순환 신경망)이란?

인풋 아웃풋 예시#2

- 인풋 : 지속적으로 변하는 수지 **주입 속도**
- 아웃풋 : **최종 공차 기준** (최종 결과 예측 데이터)



RNN의 한계 — 그래디언트 소실(Vanishing Gradient)

RNN은 원래 "데이터 길이에 상관없이 다 잘 쓸 수 있게" 만들어졌지만, 실제로는 은닉층을 여러 스텝 거치다 보면 **먼 과거의 정보가 현재 아웃풋에 잘 전달되지 않는 그래디언트 소실 문제**가 생긴. → 데이터가 짧고 최근 정보만 중요한 경우엔 RNN으로 충분하지만, 아주 긴 인풋(먼 과거도 현재에 영향을 미치는 경우)에는 범용적으로 쓰기 어려움. 이걸 극복하려고 나온 게 **LSTM**.

LSTM(Long Short Term Memory)

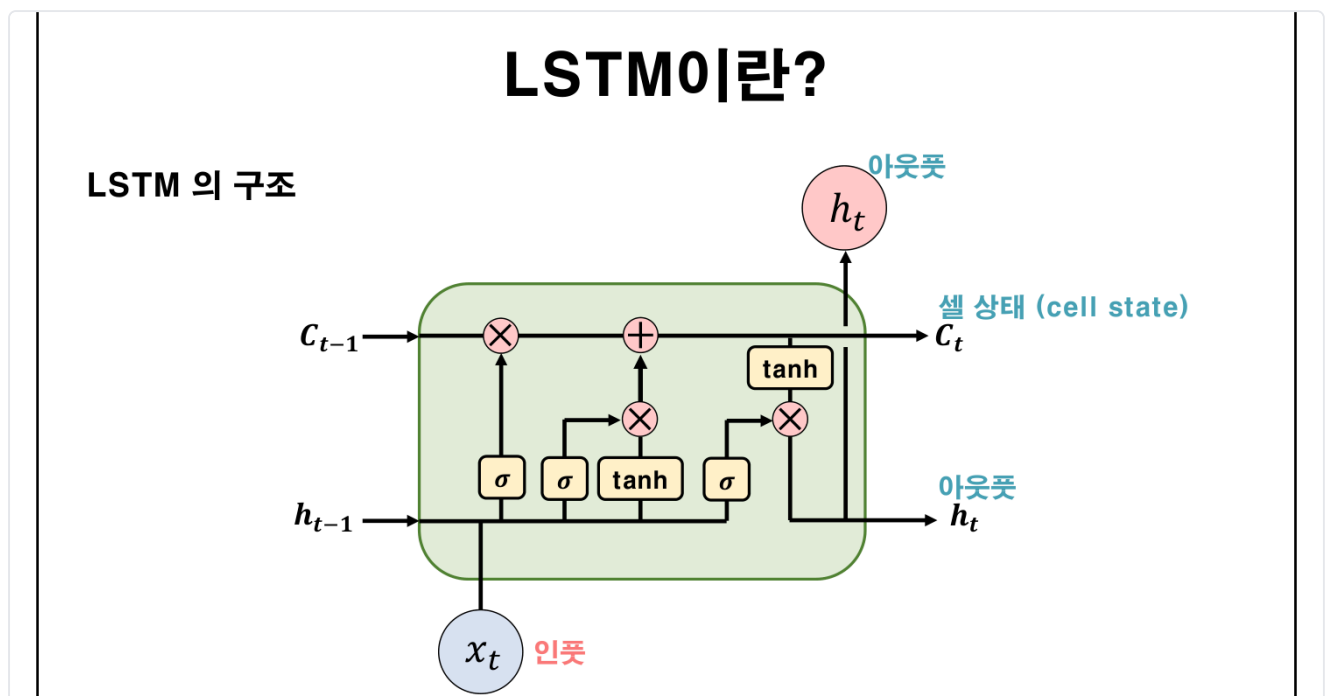
RNN 내부 네트워크를 조금 복잡하게 만들어서, **과거 정보를 다음으로 얼마나 넘겨줄지 혹은 잊어버릴지를 판단하는 구조**를 추가함. 핵심 요소는 **셀 스테이트(Cell state, C_t)** — 은닉층 정보가 바로 아웃풋에 영향을 주던 기존 RNN과 달리, 셀 스테이트를 통해 과거 정보를 계속 더해가는 구조.

LSTM 내부 구성 요소(σ =시그모이드로 0~1 값, $\tanh$ =-1~1 값):

- **Forget gate** (f_t): 과거 정보를 얼마나 기억할지 결정. $f_t = \sigma(w_f \cdot [h_{t-1}, x_t] + b_f)$ — f_t 가 0에 가까우면 과거 정보를 거의 다 잊고, 1에 가까우면 과거 정보를 최대한 유지.
- **Input gate** (i_t , $\tilde{C}_t$): 현재 정보를 얼마나 반영할지. $i_t = \sigma(w_i \cdot [h_{t-1}, x_t] + b_i)$, $\tilde{C}_t = \tanh(w_c \cdot [h_{t-1}, x_t] + b_c)$
- **셀 스테이트 업데이트**: $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$ (과거와 현재 정보가 조합됨)
- **Output gate** (o_t , h_t): $o_t = \sigma(w_o \cdot [h_{t-1}, x_t] + b_o)$, $h_t = o_t * \tanh(C_t)$ (현재 아웃풋 생성)

즉 옛날 정보에 시그모이드로 0을 곱하면 "다 잊어라", 1을 곱하면 "최대한 유지해라"는 의미이고, 이 비율(f_t , i_t)을 학습을 통해 LSTM이 알아서 조절함. 이 셀라인 덕분에 훨씬 오래된 과거 정보도 필요하면 현재 아웃풋에 반영할 수 있게 됨 (예시: 사출성형 수지유동에서 몇십초 전 정보가 중요한 경우, 자연어처리에서 문장 앞부분 주어 정보가 뒷부분까지 영향을 미치는 경우 — "주어가 나왔으면 지금 또 주어가 나오면 안 된다" 같은 비유).

📖 [4강 PDF 9~12페이지 — LSTM 셀 내부구조 다이어그램($\otimes$, $\oplus$, σ , $\tanh$ 게이트들이 단계적으로 하나씩 강조되며 설명되는 5장의 슬라이드: forget gate 계산 → input gate 계산 → cell state 업데이트 → output/ h_t 계산 → 전체 조합), 그리고 사출성형 유동선단온도 예측에 적용한 LSTM 체인 다이어그램]



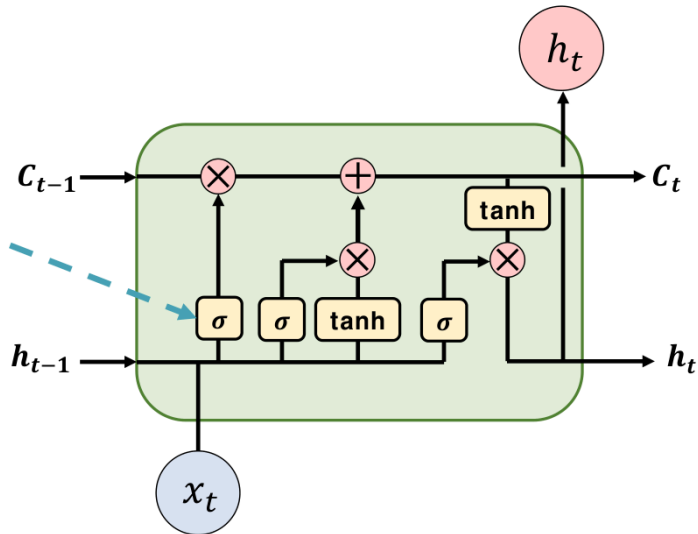
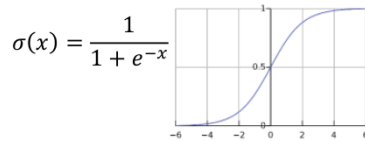
LSTM이란?

LSTM 의 구조

가중치와 바이어스 연산

$$\sigma(w_f \cdot [h_{t-1}, x_t] + b_f)$$

σ : sigmoid function

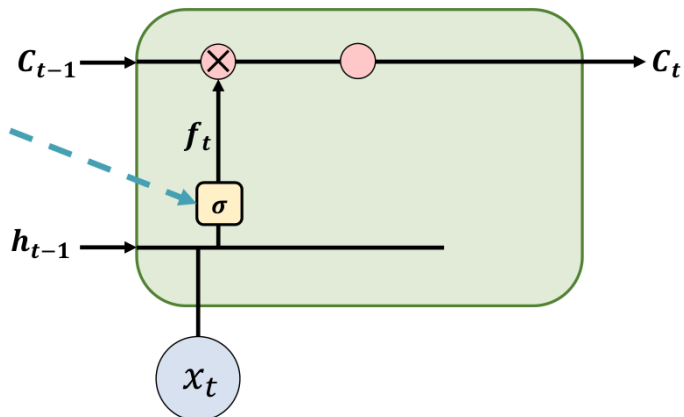


LSTM이란?

LSTM 의 구조

과거의 정보를 얼마나
기억할지 정하는 부분

$$f_t = \sigma(w_f \cdot [h_{t-1}, x_t] + b_f)$$



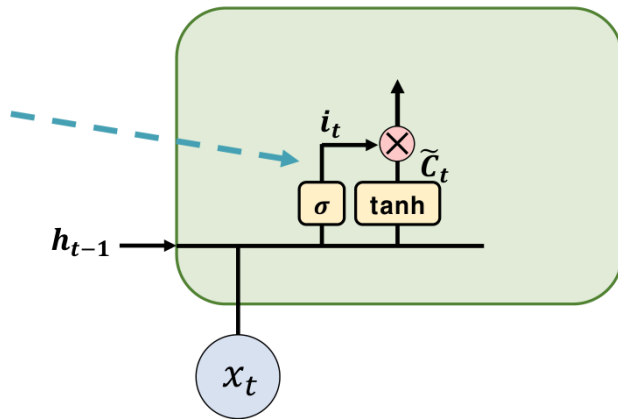
LSTM이란?

LSTM 의 구조

현재의 셀 상태 값을 계산하는 부분

$$i_t = \sigma(w_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(w_c \cdot [h_{t-1}, x_t] + b_c)$$

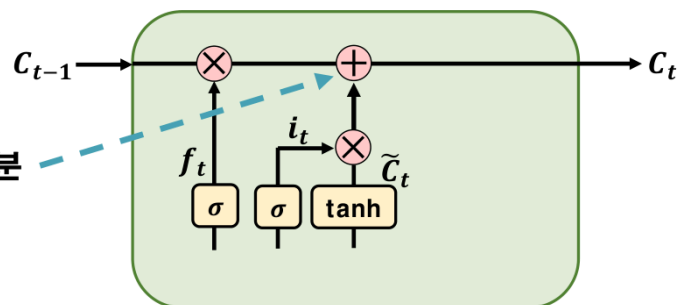


LSTM이란?

LSTM 의 구조

과거와 현재의 셀 상태가 조합되는 부분

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



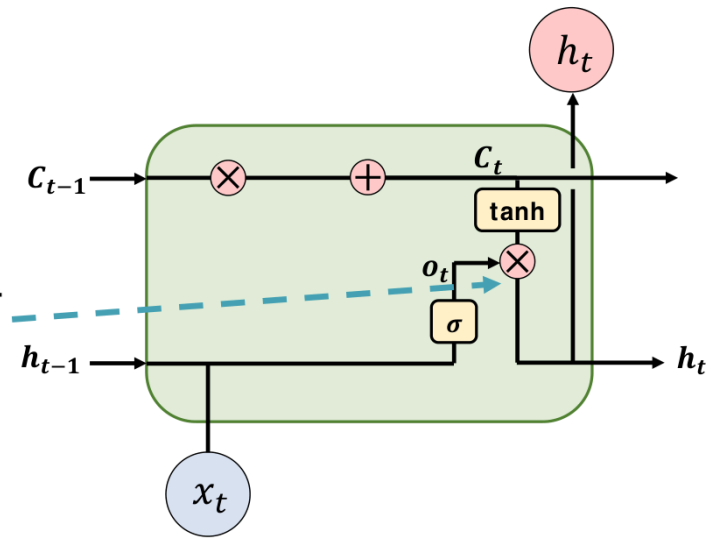
LSTM이란?

LSTM 의 구조

현재의 아웃풋(h_t)이 생성되는 부분

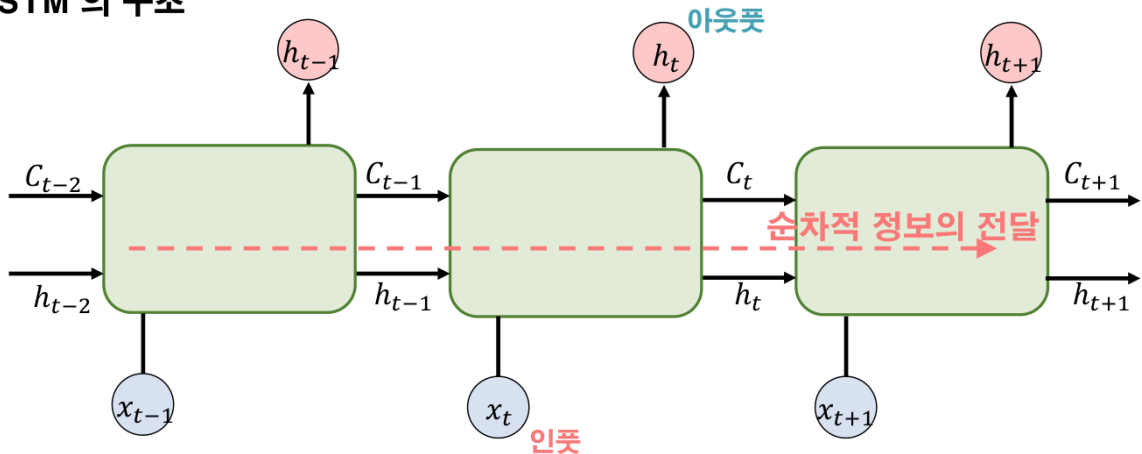
$$o_t = \sigma(w_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



LSTM이란?

LSTM 의 구조



추가 활용 예: 기계부품에 지속적으로 가해지는 하중 크기(1N→1.2N→...)를 인풋으로 넣어서 균열(crack) 진전 정도나 균열 개수를 아웃풋으로 예측하는 것도 LSTM으로 가능(고체역학/유체역학 문제에도 적용 가능하다고 언급).

LSTM 적용 사례 (KAMP)

NEW (PDF 세부수치)

- **사례 1 — Ford 엔진 정상/비정상:** 500개 센서 계측 데이터를 인풋으로. 데이터 4,921개(학습3,601/테스트1,320).

Confusion matrix: 594/87/144/495. F1 score: 0.82 (⚠️ 같은 Ford 엔진 데이터셋을 3강에서는 CNN으로 F1 0.97을 얻었는데, 여기 LSTM은 F1 0.82로 더 낮게 나옴 — 강사도 "때때로 어떤 데이터는 LSTM이 잘 되고, 어떤 경우엔 CNN이 좋기도 하다"고 직접 언급함)

- 사례 2 — 용해공정 양품/불량: 용해 온도, 교반 속도, 수분함유량을 시간별로 인풋. 데이터 83만5,200개(학습70%/테스트 30%). Confusion matrix: 2,155/785/47,955/199,655. F1 score: 0.8912. (강사 코멘트: "정상이라고 예측한 것 중 대부분 정상이지만, 비정상이라 예측한 것 중 정상이 좀 많은 아쉬움도 있다")

어떤 모델을 써야 할지 모르겠다면?

강사가 여러 번 반복해서 강조하는 메시지: "데이터셋이 주어졌을 때 LSTM도 해보고 CNN도 해보고, 완전연결 신경망도 해보고, 의사결정나무·랜덤포레스트도 해보고, 각각의 데이터에 맞게 여러 개를 다 시도해본 다음, 학습에 쓰지 않은 검증데이터로 최종 비교해서 가장 좋은 모델을 선택하라." 어떤 모델이 어떤 문제에 왜 잘 맞는지는 아직 해석이 어려운 영역이라고도 언급.

요약

- RNN: 데이터가 순서/시간 흐름이 있을 때(과거→현재→미래 순으로 영향을 주는 구조) 이를 활용하기 위해 개발됨
 - RNN은 원래 데이터 길이 무관하게 쓰려 했지만, 실제로는 과거 정보가 gradient vanishing 문제로 점점 희미해짐
 - 이 단점을 보완하려고 LSTM(Long Short Term Memory)이 등장, 셀 스테이트로 과거 정보를 얼마나 반영할지(1을 곱할지, 0을 곱할지, 그 중간을 곱할지) 학습 가능
 - LSTM이 일반 RNN보다 순차 데이터에서 훨씬 많이 쓰이고 유용한 구조
-

[5강] 오토인코더(Autoencoder)

강사: 카이스트 기계공학과 유승화 교수. 지난 4강(RNN/LSTM)에 이어서 딥러닝의 또 다른 방법인 오토인코더를 설명하고, 제조 데이터와의 연관성까지 이어서 설명하는 게 이번 강의 목적. 강의자료를 준비해준 연구실 이재민 박사과정에게 감사 인사로 시작함(음성 스크립트 기준 이름 — ⚠️ 정확한 한자/영문 표기는 확인 필요).

오토인코더(Autoencoder)라는 이름 자체가 "오토(스스로, 저절로)" + "인코더(암호화하다)"의 합성어라, 이름만 보면 "자기 스스로를 암호화하는 것" 정도로 추측할 수 있음 — 실제로 딱 그런 기능을 하는 모델이라고 강사가 도입부에서 짚어줌.

지도학습 vs 비지도학습 재정리 (오토인코더의 위치)

여태까지 배운 건 전부 **지도학습**(정답 라벨이 있는 데이터로 학습)의 일종이었기 때문에, 그 반대 개념인 **비지도학습**을 대비시켜서 설명한 다음 오토인코더로 넘어감.

지도학습 복습: 정답이 있는 데이터를 활용해서 학습하는 방식. 분류(classification)와 예측(prediction/regression) 두 기능을 함.

- **분류 예시 — 머신비전:** 이미지 데이터를 여러 개 학습시킨 다음, 사진만 보고도 양품/불량품, 불량이면 어떤 종류의 결함인지(Void, Fold 등)를 판별. 딥러닝이 많이 쓰이는 대표적인 분야.

- **예측(회귀) 예시 — 복합재(컴파운드) 설계:** 모재, 충전제, 그리고 충전제-모재 결합/분산을 좋게 하기 위한 화학적 첨가제 등을 입력치로 넣었을 때, 10가지 물성을 예측하는 걸 심층신경망이나 랜덤포레스트로 수행 가능. 반대로 "원하는 강성·인성·밀도 같은 목표 물성이 있을 때 어떤 조합의 컴파운드를 써야 하는가"라는 **역설계 문제**로도 치환 가능하다는 점도 언급.

비지도학습: 정답 라벨이 없는 데이터에서 패턴/형태를 찾아서 새로운 데이터의 결과를 예측하는 것.

- **군집화(Clustering) 예시:** 제품 몇십 개를 주고 "이 제품의 열 정도는 얼마, 강도는 얼마"처럼 라벨(특징값) 두 개만 준 다음 산 포도를 그렸을 때, "이걸 세 가지 제품군으로 분류해봐라" 혹은 "두 가지로 분류해봐라" 하는 식으로, 정답 없이 사용자가 원하는 개수의 군집으로 묶는 작업. 이런 작업을 하는 게 **군집화**.

- **오토인코더:** 비지도학습의 또 다른 대표 사례. 들어가는 것과 나가는 것이 비슷하고, 가운데는 뭔가 더 작은 구조 — 인코더 부분과 디코더 부분으로 구성된 심층신경망. 입력은 숫자 나열일 수도, RNN에서 쓰는 시계열 데이터일 수도, 이미지 데이터일 수도 있음. 핵심은 **원래보다 더 적은 숫자로 데이터를 압축했다가, 그걸로 원래 데이터를 복원한다**는 것.

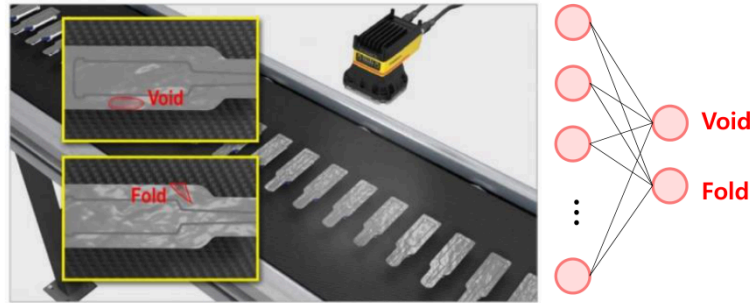
RNN 강의(4강)에서 미리 보여준 예고편: 빨간색 데이터가 들어갔을 때 그거랑 똑같이 복제하는 파란 출력이 나오는 예시가 있었는데, 이게 바로 LSTM 같은 RNN 계열 모델로 다양한 "정상파형"이 들어갔을 때 그걸 그대로 복사하도록 학습시킨 오토인코더였음(정답 라벨 없이, 그냥 다양한 정상 작동 패턴을 계속 넣어서 복사하게 학습). 이렇게 학습해두면, 나중에 정상이 아닌 이상한 데이터를 넣었을 때는 학습 안 한 패턴이라 복원을 잘 못하고 에러가 커지는데, 이 에러를 보고 "지금까지 학습한 데이터랑 다른 비정상 데이터구나"를 판별할 수 있음 — 이게 오토인코더 활용의 핵심 아이디어.

 [5강 PDF 1~2페이지 — 코그넥스 컨베이어벨트 결함(Void/Fold) 분류 이미지, 복합재 물성 예측 산점도 6종 + 의사결정나무 기반 추천 물성조합 표, 군집화 산점도, 오토인코더 인코더-잠재변수-디코더 기본 구조도]

1. Autoencoder의 정의

지도 학습(Supervised learning) : 정답이 있는 데이터를 활용해 데이터를 학습시킴

Ex) 분류(Classification)



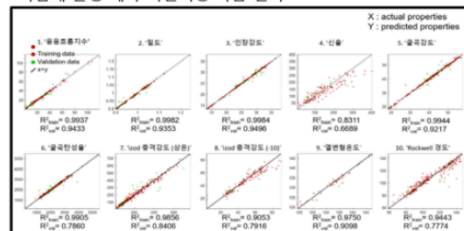
<출처: 코그넥스>

1. Autoencoder의 정의

지도 학습(Supervised learning) : 정답이 있는 데이터를 활용해 데이터를 학습시킴

Ex) 예측(Regression)

복합재 물성 예측 머신러닝 학습 결과



- 주어진 소재 조합에서 물성치 예측 성공
- 목표 물성치를 가지는 복합재 조합 예측 성공
- 특허출원 10-21-0159046 (21년 11월 18일)

Decision-Tree 알고리즘이 추천한 물성 조합

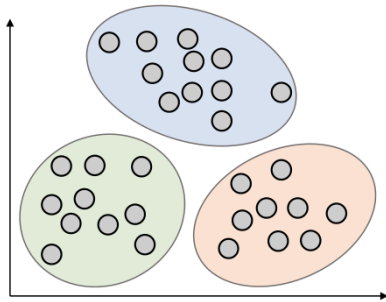
1. EP440G 비율 (%)	54	11. KCMAP-400	Filter의 비율 (%)	0
2. JX380 비율 (%)	10	12. JF3CA	Filter의 비율 (%)	0
3. E395G 비율 (%)	7	13. NAM-1	Filter의 비율 (%)	0
4. EP440G 비율 (%)	1	14. MGS4SE	Filter의 비율 (%)	0
5. EG-4180 비율 (%)	9	15. S604G	Filter의 비율 (%)	10
6.	0	16. QW50	Filter의 비율 (%)	0
7. KC-5000LC	Filter의 비율 (%)	17. GTC-501	실리콘 MIB (순도) (%)	0
8. KCM-6300C	Filter의 비율 (%)	18. ARMOCLIP-CP	Amide기 (순도) (%)	1.1
9. KC-3000	Filter의 비율 (%)	19. NA-27	액제의 비율 (%)	0.75
10. KC-2000	Filter의 비율 (%)			



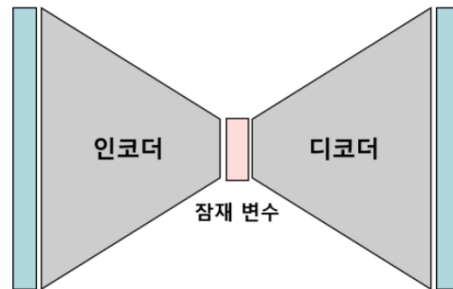
1. Autoencoder의 정의

비지도 학습(Unsupervised learning) : 정답 라벨이 없는 데이터에서 패턴이나 형태를 찾아 새로운 데이터의 결과를 예측

Ex) 군집화(Clustering)



Ex) 오토인코더(Autoencoder)



오토인코더의 정의

오토인코더: "오토(스스로)" + "인코더(암호화)"를 합쳐서, **입력과 출력이 같은(스스로 같은) 구조의 신경망**. 원래 예를 들어 6개의 숫자가 입력으로 들어왔는데, 가운데는 2개나 3개처럼 **입력치보다 적은 숫자로 전체 데이터를 압축했다**가, 이걸 다시 원래대로 복원하는 기능을 가진 게 오토인코더.

학습할 때는 입력과 출력이 같은 숫자(이미지)로 나와야 함. 손실함수는 "원래 데이터를 완벽히 똑같이 복원하기는 힘드니까(예: 원래 100개 데이터로 표현되던 걸 10개로 압축했으면 완벽 복원은 불가능), 최대한 많이 복원할 수 있게" 정하는 것 — 이 손실함수를 최소화하도록 딥러닝 노드들의 W(계수), B(상수)를 훈련시키는 게 오토인코더 학습.

왜 잠재공간의 숫자가 입력치와 똑같으면 안 되는가: 만약 가운데(잠재공간)의 숫자 개수가 입력과 똑같으면, 인공신경망이 학습될 때 그냥 다른 커넥션은 다 무시하고 "X1은 H1이랑 똑같고, H1은 또 X1해지랑 똑같다"는 식으로 **그냥 완전히 복사만 해버리면 되기 때문에** 아무 의미가 없음. 그래서 오토인코더라고 할 때는 가운데 적은 숫자로 데이터를 표현하도록 **원래보다 숫자가 작아야** 의미가 있음.

용어 정리:

- **잠재변수(Latent variable)**: 병목층(bottleneck)/히든레이어에서 얻어지는, 원래 데이터의 유의미한 특징을 나타내는 적은 개수의 숫자들
- **잠재공간(Latent space)**: 이 잠재변수들로 이루어진 공간
- **인코더(Encoder) 영역**: 입력치를 더 적은 숫자의 잠재변수로 줄여주는 왼쪽 영역
- **디코더(Decoder) 영역**: 더 작은 수의 데이터에서 원래 데이터의 숫자들을 복원하는 영역

단순하게 입력에서 바로 잠재공간으로 가는 구조 말고도, 중간에 **은닉층을 하나, 둘, 셋 더 넣어서** 더 복잡하게 구성할 수도 있음. 이렇게 하면서 "어떻게 하면 원래 들어가던 숫자들을 더 작은 숫자로 압축하고, 그걸 다시 복원할 수 있는가"를 찾아내는 게 오토인코더가 하려는 것.

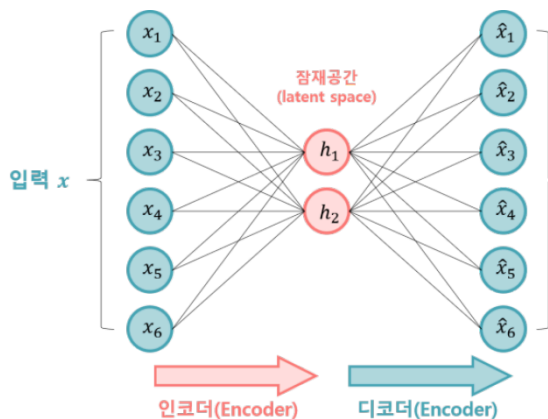
나선형(spiral) 데이터 예시: 데이터 포인트 수백 개가 두 개의 숫자(X1, X2)로 정의되는데, 사람이 보기엔 이게 원점에서 출발한 나선형(spiral) 곡선을 그리고 있다면, 원점에서부터 떨어진 거리 하나(변수 S)만으로 이 나선형 곡선 위의 모든 데이터를 표현할

수 있음. 즉 X1, X2 두 개 숫자 대신 S라는 새 변수 하나로 표현 가능하다는 걸 사람은 직관적으로 알지만, 컴퓨터는 이것 스스로 찾아내기 어려움 — 오토인코더 같은 차원축소 기법을 쓰면 컴퓨터도 이런 걸 찾아낼 수 있음.

📷 [5강 PDF 3~5페이지 — 오토인코더 구조도(입력 $x_1 \sim x_6 \rightarrow$ 인코더 $\rightarrow$ 잠재공간 $h_1, h_2 \rightarrow$ 디코더 $\rightarrow$ 출력 $\hat{x}_1 \sim \hat{x}_6$, 손실함수/파라미터 수식 표시), 히든레이어 차원이 입력보다 크면 "그대로 복사"되어버림을 X 표시로 경고하는 대조 슬라이드, 나선형(spiral) 데이터에 숨은 특징 h_1 을 학습하는 그림]

1. Autoencoder의 정의

오토인코더(Autoencoder) : 입력과 출력이 같은 구조의 인공신경망 + Bottleneck hidden layer



-손실함수(Loss function)

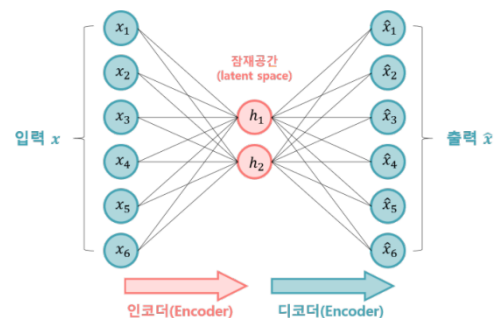
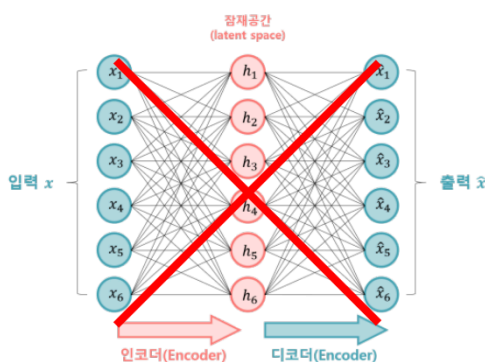
$$L = (x - \hat{x})^2$$

-학습가능한 파라미터(Learnable parameters)

$$W', b' = \underset{W, b}{\operatorname{argmin}} L$$

입력과 출력이 최대한 같도록 인공신경망을 학습

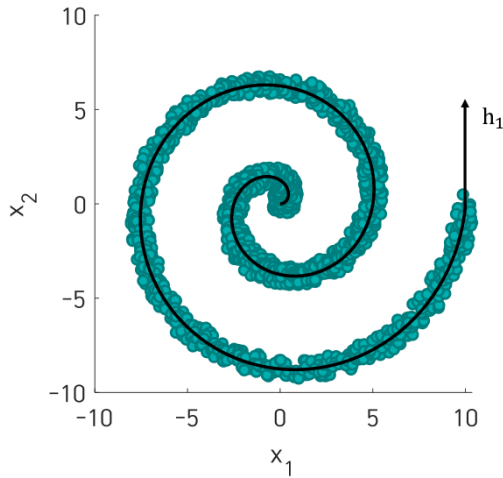
1. Autoencoder의 정의



히든레이어(Hidden layer)의 차원이 입력의 차원보다 크거나 같을 경우 입력이 그대로 복사될 수 있음 $x = h = \hat{x}$

입력보다 차원이 낮은 Bottleneck hidden layer를 구성하여 데이터의 유의미한 특징을 학습 **학습 X**

1. Autoencoder의 정의



데이터에 숨겨진 특징(잠재변수)들을 학습하여
이를 통해 차원축소, 데이터 생성, 보간, 차원축소 등의
다양한 목적으로 활용됨

오토인코더를 활용한 차원축소

왜 차원을 축소해야 하나: 이미지나 영화 같은 데이터를 컴퓨터에 저장할 때 데이터가 적을수록 당연히 좋고, 차원이 적을수록 데이터를 분류하는 작업도 더 쉬워짐. 데이터가 1차원(변수 하나)으로 표현될 땐 두 그룹(빨강/파랑)을 가르는 분류면이 굉장히 명확한데, 2차원만 돼도 분류면을 그릴 수 있는 방법이 여러 가지로 늘어남. 3차원, 4차원, 나아가 변수가 10개·100개 이상이 되면 분류면이 얼마나 복잡해질지 상상하기 어려움 — 그래서 모델 성능 향상과 저장공간 효율을 위해 **데이터 차원은 낮출 수 있으면 낮추는 게 좋음**.

MNIST 손글씨 숫자 예제: 사람들이 손으로 쓴 0~9 숫자 데이터셋. 실제 숫자 종류는 10가지(0~9)뿐이지만, 이걸 몇천~만 명이 쓴다고 하면 "8을 쓰는 방식"만 해도 수천~수만 가지 필체가 존재함. 이 손글씨를 컴퓨터에서 28×28 픽셀(흰 점=0, 검은 점=1)로 표현하면 784개(28×28)의 숫자로 하나의 이미지를 저장하게 됨. 그런데 실제로는 0~9 딱 10가지 숫자만 쓴 거니까, **784개 숫자 대신 10개 숫자의 조합만으로 표현할 수 있지 않을까?** — 이 아이디어를 오토인코더로 구현함.

만 개에 달하는 손글씨 데이터를 784차원 그대로 넣어서 전부 연결된 네트워크로 만들고, 가운데 **10개**만 남기고 다시 784개로 복원하는 오토인코더를 학습시키면, 실제로 10개 숫자의 조합만으로도 수많은 사람이 각자 다른 방식으로 쓴 0~9 숫자들을 표현할 수 있게 됨. 물론 원래 784개를 다 쓰지 못했으니 완벽하게 복원은 안 되지만, 눈으로 봤을 때 "같은 사람이 쓴 필체구나" 알아볼 수 있을 정도로는 복원됨.

NEW (강의자료 참고) 지금 보여주는 건 단순 심층신경망 구조지만, 원래 그림 데이터였으니 컨볼루션(CNN)을 인코더에, 디컨볼루션(역CNN)을 디코더에 써서 이미지 형태로 넣고 받는 구조도 가능하다고 언급함.

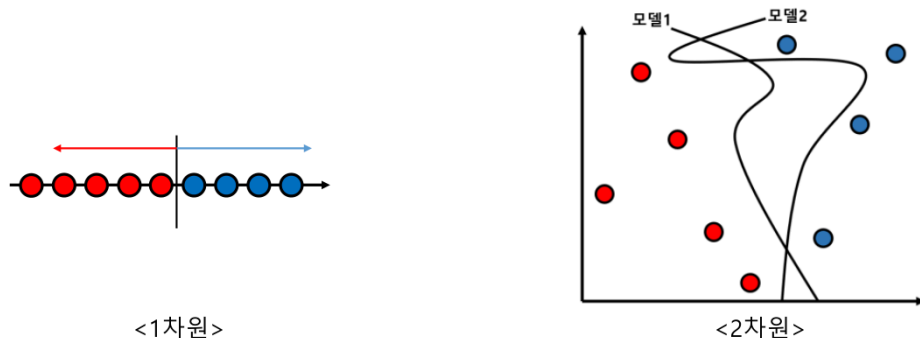
- **인코더만 떼서 쓰면:** 784차원 데이터를 10차원(잠재변수)으로 축소하는 용도로 활용 가능
- **디코더만 떼서 쓰면:** 숫자 10개(잠재변수)만으로 원래 이미지를 복원 가능 — 실제로 "8"을 쓴 사람의 데이터가 10개 숫자로 압축됐다가 디코더를 통해 나가면 원래와 유사한 "8"로, "9"."3"을 쓴 사람의 데이터도 각각 유사한 필체로 복원되는 예시가 강의자료에 제시됨

 [5강 PDF 6~8페이지 — 1차원/2차원 데이터의 성김 현상 비교 그림, MNIST 숫자 이미지(4,1,0,7,8,2,7) 28×28 픽셀 → 784 차원 벡터 변환 예시, 784-200-100-10-100-200-784 오토인코더 구조도, 그리고 인코더/디코더 각각 떼어서 쓰는 그림(입력

2. Autoencoder를 활용한 차원축소

차원의 저주 (Curse of Dimensionality)

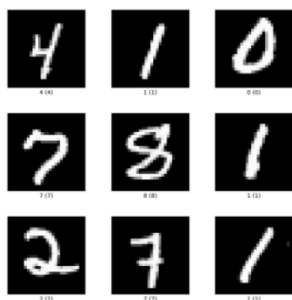
- 데이터의 차원이 높아질수록 데이터 사이의 거리가 멀어져 성김 현상(Sparsity)이 발생하고 모델의 성능이 저하됨



- 머신러닝 모델의 성능 향상과 저장 공간의 효율성을 높이기 위해 데이터의 차원은 낮추는 것이 좋음

2. Autoencoder를 활용한 차원축소

예제 : MNIST 숫자 손글씨 데이터



28 × 28 Pixel로 구성

0	0	0	0	1	1	0	0	0	0
0	0	0	1	0	0	1	0	0	0
0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	1	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	0	0	1	0	0	0
0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	1	1	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	0	0	0	0	0	0

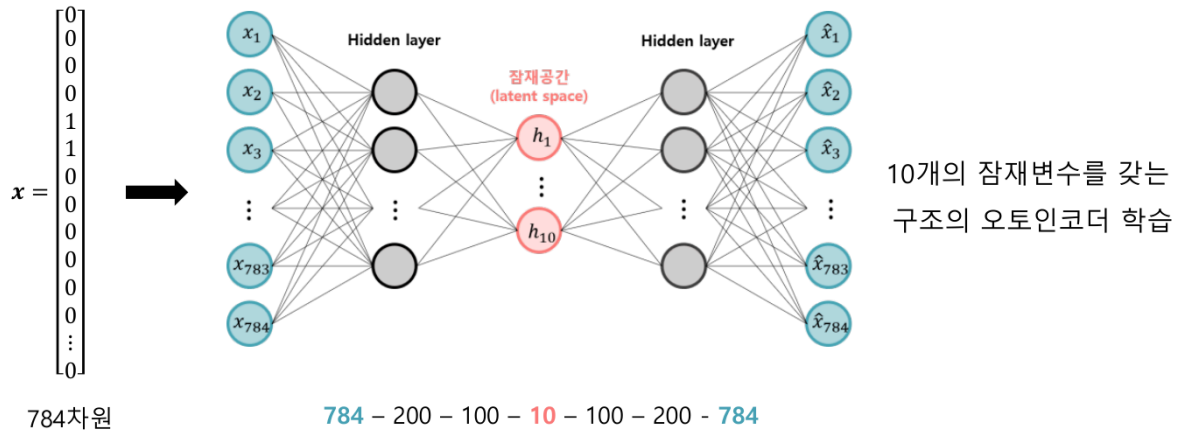


$$x = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

784차원

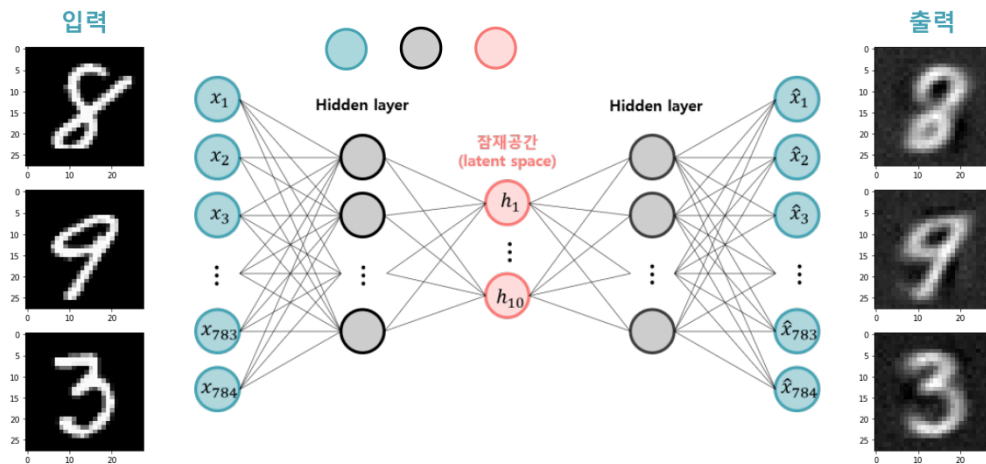
2. Autoencoder를 활용한 차원축소

예제 : MNIST 숫자 손글씨 데이터



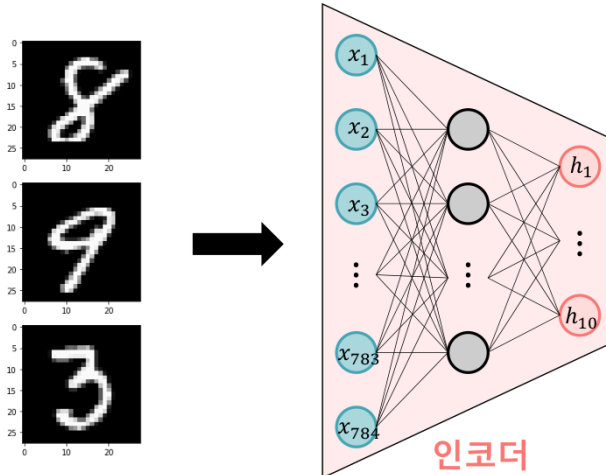
2. Autoencoder를 활용한 차원축소

예제 : MNIST 숫자 손글씨 데이터



2. Autoencoder를 활용한 차원축소

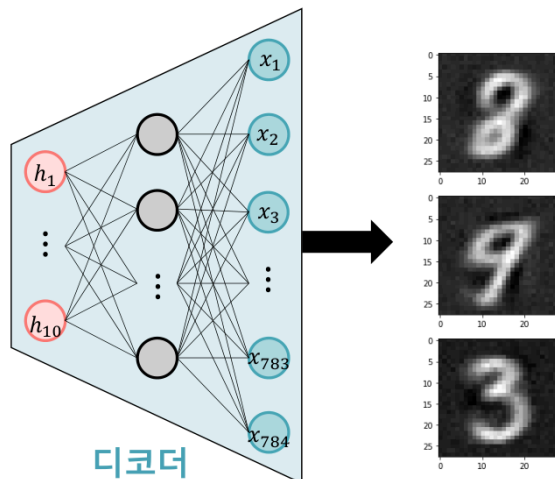
예제 : MNIST 숫자 손글씨 데이터



학습된 오토인코더의 인코더 부분을 통해
784차원의 데이터를 10차원으로 축소함
잠재변수

2. Autoencoder를 활용한 차원축소

예제 : MNIST 숫자 손글씨 데이터



학습된 오토인코더의 디코더 부분을 통해
숫자 10개만으로 이미지 복원 가능
잠재변수

오토인코더를 활용한 데이터 생성 (잠재공간 보간)

오토인코더는 데이터의 조합에 조금씩 노이즈를 줘서 새로운 데이터를 추가로 생성하는 것도 가능함. 예를 들어 0~9 숫자별 잠재변수($h_1 \sim h_{10}$) 조합에서, "1을 나타내는 잠재변수 조합" 근처에서 살짝씩 값을 튜닝하면(노이즈를 주면), "1이긴 한데 원래 쓴 사람 것과는 조금 다른 1"이 나올 수 있음.

보간(interpolation) 예시 — 두 개의 "8" 합성하기: 어떤 사람이 쓴 "8"(데이터1)과 또 다른 사람이 쓴 "8"(데이터2)이 있다고 하면:

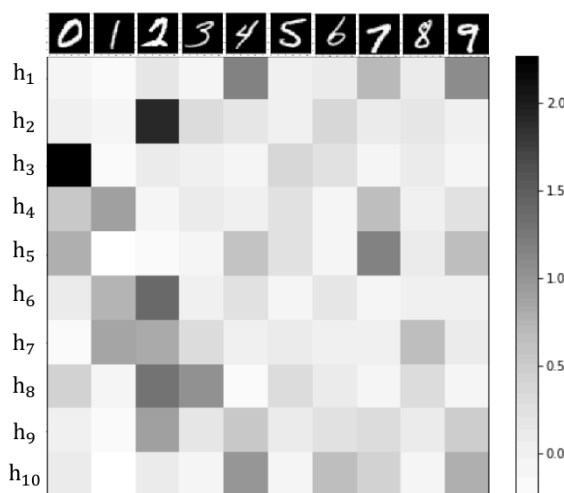
- **단순 보간:** 그냥 784개 픽셀(0/1) 값을 각각 더해서 2로 나누는 방식. 결과는 두 이미지가 겹쳐 보이면서 원래 둘 다 흰 부분은 희게, 하나만 검은 부분은 회색으로 나오는 식으로 꽤 엉뚱한 모양이 나옴.

- **잠재공간 보간**: 두 "8" 데이터를 각각 인코더에 통과시켜 잠재변수(예: 10개 숫자 조합)로 압축한 다음, 이 **잠재변수 두 개를 더 해서 평균**낸 뒤 디코더로 복원. 이렇게 하면 원래 두 "8"과는 다르지만, 단순 보간처럼 엉뚱하지 않고 **자연스러운 새로운 필체의 "8"**이 나옴.

핵심은 "압축된 공간, 즉 데이터의 정보를 많이 담고 있는 잠재공간에서 새로운 데이터를 만들면 훨씬 더 양질의(리얼한) 새 데이터를 생성할 수 있다"는 것. 단순히 원본 픽셀 값을 평균 내는 것과는 결과가 확연히 다르다는 게 이 예시의 핵심 포인트.

 [5강 PDF 9~11페이지 — 숫자별(0~9) 잠재변수 h1~h10 히트맵, 데이터1("8")/데이터2("8" 다른 필체) 원본 이미지와 각각의 잠재변수 벡터값, h_new 평균 계산 및 디코더로 복원된 결과 이미지, 그리고 "단순보간"(흐릿하게 겹친 이미지) vs "잠재공간보간"(선명한 새 숫자 이미지) 비교 4분할 그림]

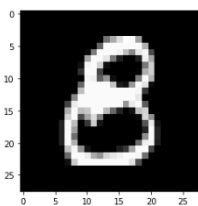
2. Autoencoder를 활용한 차원축소



잠재 공간에서의 보간을 통해
데이터 추가 생성 가능

2. Autoencoder를 활용한 차원축소

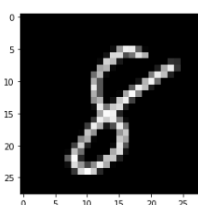
<데이터1>



$x_1 =$



<데이터2>

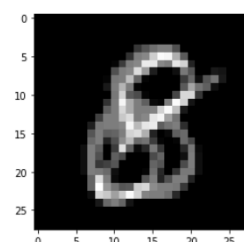


$x_2 =$

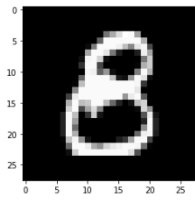


$$x_{\text{new}} = \frac{x_1 + x_2}{2} =$$

<단순보간결과>



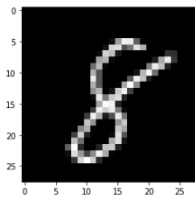
2. Autoencoder를 활용한 차원축소



$$h1 = [0.2 \quad 0.1 \quad 0.1 \quad -0.2 \quad 0.3 \quad 0.1 \quad 0.2 \quad 0.1 \quad -0.1 \quad 0.2]$$



$$h_{new} = \frac{h1 + h2}{2} = [0.15 \quad 0.15 \quad 0.1 \quad -0.2 \quad 0.35 \quad 0.15 \quad 0.25 \quad 0 \quad 0 \quad 0.15]$$

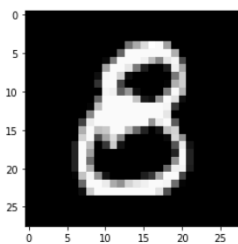


$$h2 = [0.1 \quad 0.2 \quad 0.1 \quad -0.2 \quad 0.4 \quad 0.2 \quad 0.3 \quad -0.1 \quad 0.1 \quad 0.1]$$

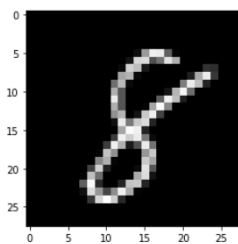
잠재공간에서 보간을 통해 새로운 데이터 생성

2. Autoencoder를 활용한 차원축소

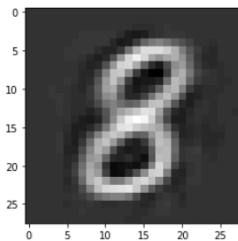
<데이터 1>



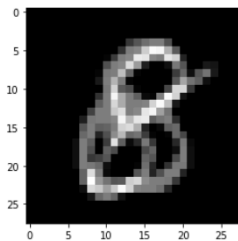
<데이터 2>



<잠재공간보간>



<단순보간>



잠재공간이 추출된 중요한 특징이므로

이 공간에서 보간하면

훨씬 더 양질의 데이터 생성 가능

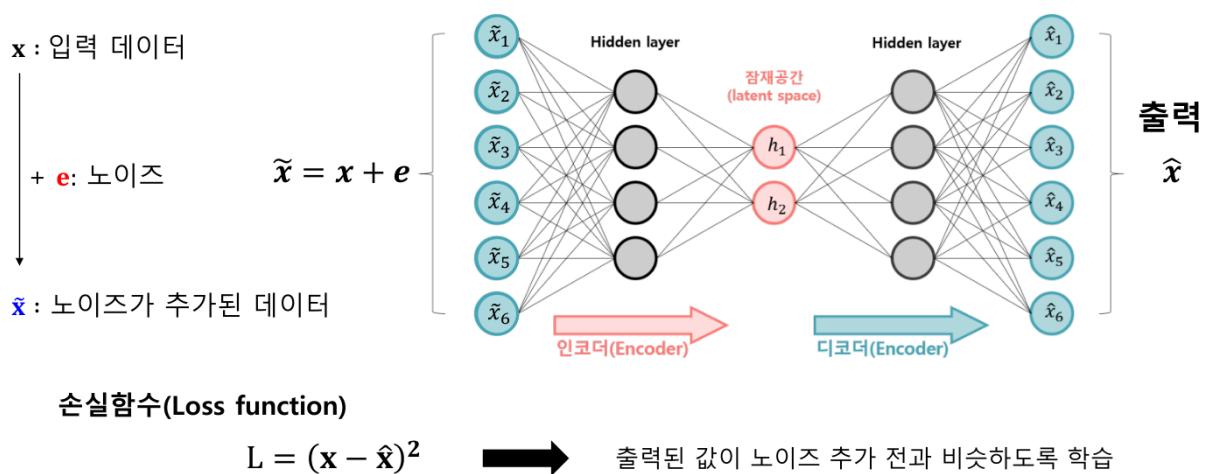
오토인코더를 활용한 노이즈 제거

오토인코더의 대표적인 응용 중 하나가 **잡음(노이즈) 제거**. 원래 숫자 6개로 표현되는 데이터가 있다고 하면, 여기에 노이즈를 각 숫자마다 조금씩 추가한 "노이즈 낀 입력 데이터"를 많이 만들고, **출력은 노이즈가 제거된 원본 데이터가 나오도록** 손실함수를 정해서 학습시킴. 즉 "정확히 원래 데이터를 복원"하는 게 아니라 "원래 데이터 + 노이즈가 입력으로 들어와도, 출력에서는 노이즈가 빠진 원본이 나오도록" W, B를 훈련시키는 것. 이렇게 학습하면 입력은 노이즈 있는 상태, 출력은 노이즈 없는 상태로 나오는 오토인코더가 만들어짐. 오토인코더는 입력과 출력의 크기가 같기 때문에 이런 응용이 가능함.

예제 — 기계 소음 신호: 기계가 소음으로 인해 고장났는지 판별하고 싶은데, 노이즈 때문에 신호가 자글자글하게 훼손된 상황을 가정. 원래 신호를 0.01초~5초까지 벡터화하면 깔끔한 숫자열이 나오는데, 여기에 노이즈(역시 0.01초~5초 구간의 잡음)를 더하면 숫자가 지저분해짐. 이 노이즈 낀 신호를 입력으로, 노이즈 없는 원본을 출력으로 하는 오토인코더를 훈련시키면, 나중에 새로운 노이즈 낀 신호를 넣었을 때 **노이즈가 걸러진 깨끗한 신호를 출력**하게 됨. 단, 이런 훈련에는 잠재공간으로 들어가는 W, B 파라미터가 굉장히 많아서 **많은 양의 데이터가 필요**하다는 점도 언급됨.

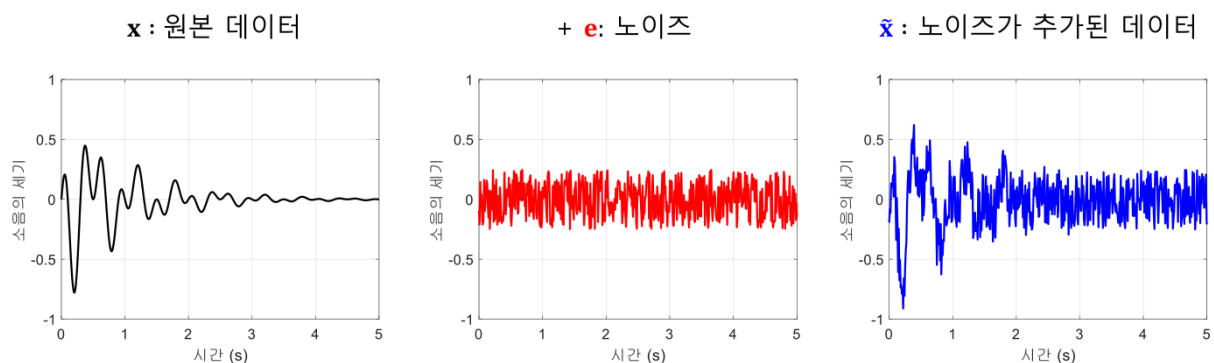
📷 [5강 PDF 12~15페이지 — 노이즈 제거 오토인코더 구조도($\tilde{x}=x+e$), 원본 신호/노이즈/노이즈 추가된 신호 3종 그래프, 신호를 500차원 벡터로 표현하는 과정, 그리고 노이즈 낀 신호를 넣었을 때 노이즈가 제거된 신호가 나오는 최종 결과 비교 그래프]

3. Autoencoder를 활용한 노이즈 제거



3. Autoencoder를 활용한 노이즈 제거

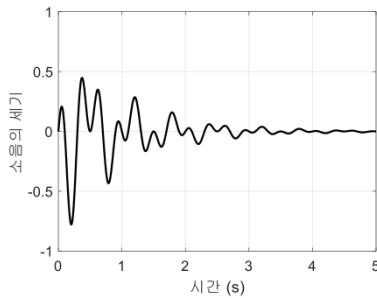
예제) 기계의 소음으로 고장인지 판별하고 싶은데
노이즈로 인해 데이터가 훼손되었음



3. Autoencoder를 활용한 노이즈 제거

예제) 기계의 소음으로 고장인지 판별하고 싶은데
주변 노이즈로 인해 데이터에 노이즈가 추가되었음

x : 원본 데이터



신호의 벡터화



$x =$

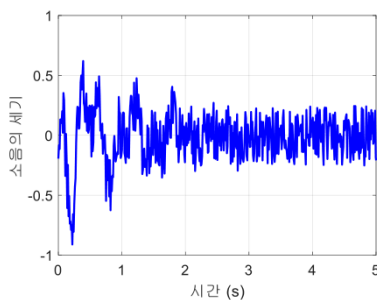
0	→ : 0.01초에서 소음의 세기
0.05	→ : 0.02초에서 소음의 세기
0.10	→ : 0.03초에서 소음의 세기
0.15	
0.20	⋮
0.25	
0.20	
0.15	
⋮	
0.02	→ : 4.99초에서 소음의 세기
0.01	→ : 5.00초에서 소음의 세기

500차원의 데이터 :

3. Autoencoder를 활용한 노이즈 제거

예제) 기계의 소음으로 고장인지 판별하고 싶은데
주변 노이즈로 인해 데이터에 노이즈가 추가되었음

$\tilde{x}$: 노이즈가 추가된 데이터



신호의 벡터화



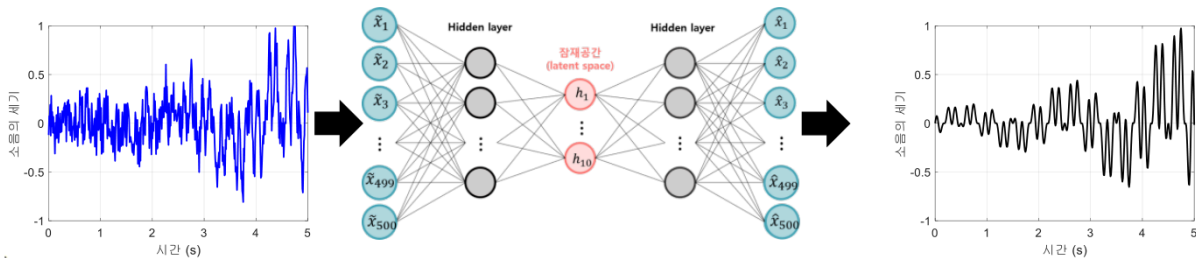
$\tilde{x} =$

0	→ : 0.01초에서 소음의 세기
0.02	→ : 0.02초에서 소음의 세기
0.11	→ : 0.03초에서 소음의 세기
0.31	
0.25	⋮
0.23	
0.27	
0.43	
⋮	
0.24	→ : 4.99초에서 소음의 세기
0.15	→ : 5.00초에서 소음의 세기

500차원의 데이터 :

3. Autoencoder를 활용한 노이즈 제거

예제) 기계의 소음으로 고장인지 판별하고 싶은데
주변 노이즈로 인해 데이터에 노이즈가 추가되었음



학습된 오토인코더에, 새로운 **노이즈 포함된 신호**를 입력하면

노이즈가 제거된 신호를 출력함.

KAMP 적용 사례 — 사출성형기 AI 데이터셋 (이상탐지)

강사가 실제로 KAMP(캄프) 예제 홈페이지에 있는 사출성형기 AI 데이터로 설명한 사례. 충전시간, 계량시간, 스크류 최대속도 등 **25개의 특성**을 센서로 계속 받아들이고 있고, 오토인코더를 이용해서 정상/불량 이상탐지를 해볼 수 있음.

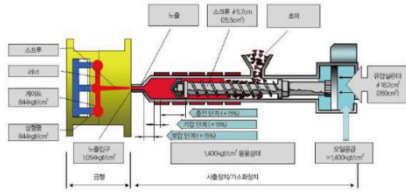
데이터 구성: 전체 데이터셋 중 정상인 경우가 **6,697개**, 사출성형기에 뭔가 이상이 난 경우가 **39개**. 오토인코더는 정상 데이터 중 일부 비율을 뽑아서 **정상 데이터 조합만 입력치로, 출력도 정상 데이터 조합만** 나오도록 학습시킴.

테스트 및 판정 원리: 오토인코더가 학습한 건 정상 데이터뿐이므로, 정상 패턴과 유사한 데이터를 넣으면 출력도 유사하게 나옴. 반면 학습에 쓰지 않았던 정상 데이터(2,697개 정도)를 넣으면 여전히 비슷하게 나오지만, **비정상 데이터(39개)를 넣으면 원래와 많이 다른 출력**이 나옴 — 머신러닝은 훈련한 데이터셋 근처에서만 잘 작동하기 때문에, 학습 안 한 비정상 패턴은 복원이 잘 안 되는 것. 실제로 정상 데이터 2,697개에 대한 평균제곱오차(MSE)는 **0.06~0.08** 수준으로 원본과 비슷하게 나오는 반면, 불량 데이터를 넣었을 때는 오차가 **0.15~0.75** 수준으로 **2배 이상 커짐**. 이 오차 크기를 보고 불량 여부를 판정할 수 있음.

왜 이 방식이 유용한가 — 불균형 데이터 문제: 기존 CNN/RNN 같은 지도학습 방법론은 불량 데이터와 정상 데이터를 동시에 갖고 있어야 하고, 그 두 클래스 비율이 어느 정도 비슷해야 학습이 잘 됨. 그런데 실제 제조 현장에서는 정상 데이터는 아주 많고 (예: 일주일 중 6일은 정상, 하루 중 1시간~5분 정도만 이상 발생), 비정상 데이터는 극히 적은(정상 만 개에 비정상 100개 이하, 혹은 그보다 더 적은) **극단적 불균형**이 흔함. 이럴 땐 비정상 데이터는 아예 잊어버리고, 오토인코더로 수많은 정상 데이터의 패턴만 학습시킨 뒤, 정상 패턴과 다른 게 들어왔을 때 입력-출력 차이가 크게 벌어지는 걸로 불량을 판별하는 방식이 훨씬 실용적이라는 게 강사의 결론.

[5강 PDF 15~16페이지 — 사출성형기 공정 흐름도, 정상 6,697개/불량 39개 데이터 구성 다이어그램, 학습된 오토인코더의 출력오차 히스토그램(정상 vs 불량 비교)]

KAMP 예제 : 사출성형기 AI 데이터셋



입력 데이터 : 충전시간, 계량시간, 스크류의 최대 속도 등의 25개의 특성



오토인코더를 통한 이상탐지

불량 VS 정상

- 데이터셋 구성

정상 데이터 : 6,697개
 불량 데이터 : 39개

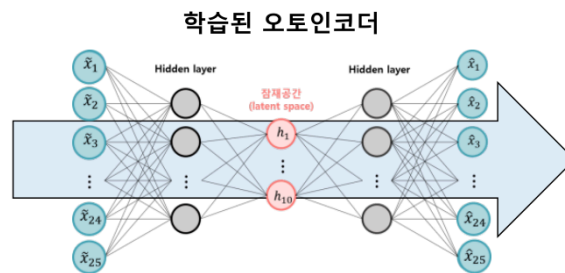
4,000개
 2,697개
 39개

- 학습 데이터 셋 : 노이즈 제거 오토인코더 모델 학습

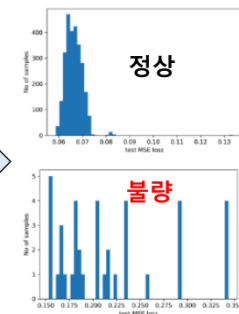
- 테스트 데이터 셋 : 학습된 모델의 평가

KAMP 예제 : 사출성형기 AI 데이터셋

- 테스트 데이터 셋



출력 오차



정상 데이터만 사용하여 오토인코더의 학습시켰기 때문에, 불량 데이터의 경우 출력과 입력 사이의 차이가 큼

이를 이용하여 출력 오차가 특정값 보다 클 경우 불량으로 판정함

5강 요약 (강사가 직접 정리한 내용)

- 오토인코더 정의:** 입력과 출력이 같은 구조의 신경망으로서, 병목층(bottleneck)에서 숫자 개수가 원래보다 작아야 함 — 차원을 축소한 뒤 그 축소된 숫자로 원래 데이터를 복원하는 인코더/디코더 구조. 차원축소, 데이터 생성, 노이즈 제거 등 다양한 목적으로 활용 가능.
- 차원축소 예시:** 784개 숫자(28×28 이미지)가 들어갔을 때 또 다른 784개로 복원하는데, 완벽 복원은 못해도 "8,9,3"처럼 원래 숫자와 필체가 유사하게 복원됨.
- 데이터 생성(보간):** 두 데이터의 잠재공간 값을 더해서 평균 낸 뒤 디코더로 복원하면 그럴싸한 새 데이터가 나오지만, 원본 픽셀(784개)을 단순히 평균 내면 엉뚱한 데이터가 나옴.

4. **노이즈 제거**: 입력에 노이즈를 더하고 출력은 노이즈 없는 원본과 같도록 학습시키면, 학습된 오토인코더에 노이즈 낀 데이터를 넣었을 때 노이즈가 빠진 데이터가 출력됨. 나아가 정상 데이터만 만 개 가량 학습시킨 뒤 비정상 패턴을 넣으면 입력-출력 오차가 커지는 것으로 **정상/비정상 판정(이상탐지)**에도 활용 가능.

강의 시리즈 전체 마무리 (강사의 최종 총정리)

5강을 마치면서 강사가 0강부터 5강까지 전체를 한 번에 되짚어줌:

- **0강**: 머신러닝이란 무엇인가 — 회귀(예측) 혹은 분류라는 기능을 하는 게 머신러닝이고, 머신러닝 모델의 성능 테스트는 학습에 쓴 데이터셋이 아니라 학습 데이터와 유사하지만 다른 새로운 데이터(검증 데이터)로 평가한다.
- **1강**: 의사결정나무 — 분류를 하는 데 중요한 머신러닝 모델이고, 정보의 불순도를 낮추는 방향으로 학습된다.
- **2강**: 심층신경망 — 사람의 신경망을 유사하게 본떠 만든 머신러닝 모델이고, 기존에 잘 풀리지 않던 학습 문제도 풀 수 있게 해준다.
- **3강**: CNN — 합성곱이 무엇인지, 합성곱을 통해 가까이 있는 데이터끼리의 관계(수직선/수평선 등)를 파악하는 필터를 학습하는 게 CNN이고, 이미지 데이터뿐 아니라 일반 신호 데이터 학습에도 잘 쓰인다.
- **4강**: RNN — 데이터의 순차적인 순서가 중요한 데이터를 학습할 수 있는 게 RNN 개념이고, 초기 RNN이 잘 안 쓰였던 이유는 오래된 과거 데이터가 현재까지 영향을 잘 못 미쳤기 때문이며, 이를 극복하기 위해 LSTM이 만들어져서 실제로 더 많이 쓰인다.
- **5강**: 오토인코더 — 지도학습처럼 정답이 있는 게 아니라, 입력치 자체가 출력이 돼야 하는 비지도학습. 차원축소, 노이즈 제거, 정상/비정상 판정(이상탐지) 등 여러 기능을 할 수 있다.

강사의 마무리 인사로 강의 전체가 끝남: "그동안 경청해주셔서 감사합니다."

☁ 정리하며 (팀 스터디용 포인트)

- 이 강의 전체를 관통하는 메시지는 딱 하나로 요약됨: **"머신러닝 = 제조데이터 + 모델 → 예측(회귀) 또는 분류이고, 그 성능은 반드시 검증(테스트)데이터로 평가해야 한다."** 이 원칙이 0강부터 5강까지 계속 반복해서 등장함(살짝 다른 표현으로 매 강의 요약마다 다시 나옴). 팀 발표나 캡스톤에서 모델 비교할 때 이 프레임을 그대로 써도 좋을 듯.
- "어떤 모델이 제일 좋은지는 데이터를 보기 전엔 알 수 없다"는 점도 반복해서 강조됨(4강에서 특히). 그래서 실전에서는 의사결정나무/랜덤포레스트/완전연결 딥러닝/CNN/RNN·LSTM을 다 시도해보고 검증데이터 성능으로 최종 선택하는 게 정석이라는 태도.
- 강의 전체에서 KAMP(중소벤처기업부, Korea AI Manufacturing Platform) 데이터셋이 계속 등장함(살균기, 사출성형, CNC, Ford 엔진, 용해공정 등). 캠프(부트캠프) 실습에서 실제로 이 KAMP 홈페이지 데이터를 갖고 의사결정나무/랜덤포레스트/CNN/LSTM 등을 직접 돌려볼 수 있다고 강사가 여러 번 언급했으니, 팀 프로젝트/경진대회 데이터 소스로 KAMP 홈페이지(kamp-ai.kr)를 확인해보면 좋을 것 같음.
- 과적합(오버피팅) 개념이 0강(예측/회귀 맥락)→2강(과소적합/과대적합 정식 용어, underfitting 쉬움 vs overfitting 어려움)→1강(의사결정나무의 대표적 단점) 순서로 계속 다른 각도에서 재등장하는 구조라, 발표자료 만들 때 이 흐름을 하나의 축으로 잡아서 스토리텔링하면 좋을 듯.
- 5강(오토인코더)의 "잠재공간 보간 vs 단순 보간" 비교는 데이터 증강(augmentation)이나 이상탐지 프로젝트에 바로 응용해볼 만한 아이디어로 보임.

🔗 참고 — 강의자료에만 있던 내용(NEW) 모아보기 + 확인 필요 사항

NEW 강의자료 전용 내용 모음

- 0강: 제조업 머신러닝 적용목적 I의 3가지 연구성과 예시(복합재 조합 자동탐색, 접착기동 최적설계, 베이징안 최적화), 적용목적 II의 4분류 정식 영문명(FDC/APC/QC-진단/Tool Operation Optimization)
- 2강: Match/Area/Blemish/Sort 4종 이미지 검사 아이콘 예시
- 3강: 파도 사진→파고 예측 KAIST 연구는 스크립트에도 언급되긴 했으나 구체 수치는 없음(정성적 설명만)
- 4강: LSTM 각 게이트(forget/input/output gate) 수식 전개가 PDF에는 단계별로 아주 상세히 나와있는데, 스크립트 설명과 거의 일치하긴 하나 PDF 쪽이 수식 표기가 더 정확함(스크립트는 구어체라 수식을 말로 풀어서 설명함)
- **5강:** 추가로 음성 스크립트를 확보해서 본문을 스크립트 기반으로 다시 작성함(이전 버전은 PDF 슬라이드만으로 정리됐었음). 특허출원번호 "10-21-0159046"(복합재 물성 예측 관련, PDF 원문 그대로)은 여전히 PDF 슬라이드 전용 정보.

⚠ 확인이 필요한 사항 (음성인식 오류/불확실 고유명사·수치 모음)

- 회사명 "SCT"(활성탄 재생 회사), "아주스틸"(컬러강판 회사) — 음성 텍스트라 정확한 회사명 표기인지 확인 필요.
- 강사 성함 표기가 스크립트 안에서 "유승하 교수"/"유승화 교수"/"유성화 교수" 등으로 매번 다르게 적혀있었음 — PDF 표지 기준으로 **"유승화 교수"**가 맞는 것으로 확인함.
- 조교/도움 주신 분 성함: "정지영 박사과정"(1강, 4강 관련), "이태국 박사과정"(2강, 3강 관련) — 음성인식 표기라 정확한 한자/영문 스펠링 확인 권장.
- 1강 살균기 사례의 confusion matrix 숫자: 스크립트 음성 설명("정상 2300개, 비정상 196개, 비정상 예측 중 정상 1개·비정상 237개")과 PDF 표에 적힌 숫자(237/196/1/2381)의 행·열 대응이 정확히 일치하는지 애매함 — 팀 스터디 자료에 인용할 땐 PDF의 원본 수치(237, 196, 1, 2381)를 기준으로 쓰고 어느 셀이 무엇인지는 원본 PDF 표를 다시 확인하는 걸 추천.
- 4강에서 Ford 엔진 데이터셋에 대해 CNN(3강, F1 0.97)과 LSTM(4강, F1 0.82) 결과가 다르게 나온 것 — 강사도 "경우에 따라 CNN이 나올 때도, LSTM이 나올 때도 있다"고 직접 코멘트했지만, 정확히 같은 데이터셋/같은 분할인지(둘 다 4,921개, 학습

3,601/테스트1,320으로 표기가 동일)는 원본 자료 재확인 권장.

6. 특허출원번호 "10-21-0159046"(5강 PDF, 복합재 물성 예측 관련) — PDF 원문 그대로 옮겼으나 자릿수 표기가 일반적인 한국 특허출원번호 형식과 다소 달라 보여 원본 재확인 권장.